

Documentación Técnica

13 de mayo de 2026

Índice

1. Clustering	5
1.1. Objetivo	5
1.2. Datos y preprocesamiento	6
1.3. Ingeniería de <i>features</i> por conductor	6
1.3.1. Conteos, velocidad media y uso de carril	6
1.3.2. Velocidad relativa	7
1.3.3. Headway (brecha temporal)	7
1.3.4. TTC y tasa de conflictos	7
1.3.5. Cambios de carril	8
1.3.6. Ponderación mensual (opcional)	8
1.3.7. Actividad temporal	8
1.4. Preparación de datos para clustering	8
1.5. Métodos de clustering	8
1.5.1. K-means	8
1.5.2. Gaussian Mixture Models (GMM)	9
1.5.3. HDBSCAN	9
1.6. Conductores frecuentes vs. raros (opcional)	9
1.7. GMM con clusters dinámicos	9
1.8. Salidas y artefactos	11
1.9. Variables de clusters agregadas por pórtico e intervalo	11
2. Predicción de accidentes	11
2.1. Objetivo	11
2.2. Fuentes de datos	12
2.3. Discretización temporal y notación	12
2.4. Ingeniería de variables (features)	12
2.4.1. Variables macro de flujo	12
2.4.2. Variables de clusters agregadas (opcional)	12
2.5. Definición del target (etiqueta)	13
2.6. Construcción del dataset	13
2.7. Partición temporal (train/validación/test)	13
2.8. Manejo de desbalance	13
2.9. Modelos disponibles	14
2.10. Calibración de score y umbral operacional	14
2.11. Métricas de evaluación	14
2.12. Pestaña Optuna	15

2.13. Pestaña Modelos	15
2.14. Pestaña Experimentos	15
2.15. Artefactos	16
2.16. XAI para Base+Cluster	16
3. Ingeniería de <i>features</i> con GMM dinámico y DuckDB	17
3.1. Propósito y alcance	17
3.2. Contrato de datos	17
3.2.1. Base de flujos	17
3.2.2. GMM dinámico	18
3.3. Variables base de flujo	18
3.3.1. Mapeo de categorías	18
3.3.2. Agregación por pórtico e intervalo	18
3.3.3. Deltas temporales	19
3.4. Variables de cluster: etiqueta dura versus membresía probabilística	19
3.4.1. Modo de asignación dura	19
3.4.2. Modo de membresía suave con probabilidades GMM	19
3.5. Métricas agregadas por cluster	20
3.6. Alineamiento causal del GMM dinámico	21
3.6.1. Filtración temporal	21
3.6.2. Implementación con ASOF JOIN	21
3.7. Plan de ejecución por lotes	21
3.8. Modelo de memoria y política de carga	22
3.9. Salida y compatibilidad	22
3.10. Validación y pruebas	23
3.11. Implicaciones científicas	23
4. Réplica del paper en NLP in Severity	23
4.1. Propósito y alcance	23
4.2. Protocolo bloqueado en código	24
4.3. Ejecución paso a paso	24
4.3.1. Etapa 1: Construcción del contexto de ejecución	24
4.3.2. Etapa 2: Reanudar, cargar o comenzar limpio	25
4.3.3. Etapa 3: Ruta congelada	25
4.3.4. Etapa 4: Ruta de reconstrucción raw	25
4.3.5. Etapa 5: Corridas por modalidad (M1, M2, M3)	26
4.3.6. Etapa 6: Nested CV y ajuste final	27
4.3.7. Etapa 7: Consolidación por ruta	27
4.3.8. Etapa 7b: Ruta de actualización de embeddings (opcional)	28
4.3.9. Etapa 8: Comparación entre frozen y raw	28
4.3.10. Etapa 9: Staging y promoción a LaTeX	28
4.3.11. Etapa 10: Sincronización al registry	28
4.4. Contraste con el manuscrito	29
4.5. Estructura de checkpoints y artefactos	30
4.6. Cómo leer una corrida	30

5. Drift detection	31
5.1. Objetivo, alcance y relación con el paper	31
5.2. Dependencias y fuentes de datos	31
5.3. Mapa de la interfaz tab a tab	31
5.4. Feature engineering y construcción del target	32
5.5. Preparación de datos: Stage 1 y Stage 2	36
5.6. Selección de variables	36
5.7. Diseño experimental y estrategias	36
5.8. Neural drift: objetivo, alcance y unidad experimental	42
5.9. Pipeline operacional paso a paso	42
5.10. Dataset temporal, augmentación y construcción de ventanas	43
5.11. Configuración y parámetros principales	44
5.12. Modelos predictivos, detector neuronal y drift channels	46
5.13. Strategies y optimizaciones de adaptación	47
5.14. Backtest: diseño, métricas y salidas	48
5.15. Checkpointing, reanudación y ejecución en vivo	49
5.16. Experimentos: protocolo fijo, fases, score y criterio de ganador	50
5.17. Validación mediante pruebas	51
6. Introducción	52
7. Preprocesamiento de Datos	52
8. Cálculo de Variables (Nivel Evento)	52
8.1. Headway (h_i)	53
8.2. Velocidad Relativa Indexada ($v_{rel,i}$)	53
8.3. Conflicto y TTC (Time To Collision)	53
9. Agregación por Patante	53
9.1. Variables Agregadas	53
9.1.1. Velocidad Promedio ($\bar{v}_P$)	53
9.1.2. Velocidad Relativa Promedio ($\bar{v}_{rel,P}$)	54
9.1.3. Headway Promedio ($\bar{h}_P$)	54
9.1.4. Tasa de Conflicto (CR_P)	54
9.1.5. Uso de Carriles ($Lane_k$)	54
9.1.6. Tasa de Cambio de Pista (LCR_P)	54
9.2. Estrategias de Consolidación	54
9.2.1. 1. Agregación Global (Default)	54
9.2.2. 2. Ponderación Mensual	55
9.2.3. 3. Actividad Temporal	55
10. Definición de Conductores Frecuentes	55
11. Implementación en Python	55
11.1. Limpieza de Datos: <code>clean_flujos_for_clustering</code>	55
11.2. Cálculo de Features: <code>Clusterization</code>	56
11.2.1. 1. Métricas Simples y Pre-cálculo	56
11.2.2. 2. Velocidad Relativa	56
11.2.3. 3. Métricas de Interacción (Headway, Conflicto)	56

11.2.4. 4. Cambios de Pista (Lane Changes)	57
11.2.5. 5. Ponderación (Si aplica)	57
12. Alcance y metodología	57
13. Mapa general del pipeline	57
14. Pipeline <i>tab a tab</i> (Streamlit Graph Builder)	58
14.1. Eventos	58
14.2. Feature Engineering	58
14.3. Feature Explorer	58
14.4. Feature Selection	58
14.5. Graph	58
14.6. Visual Graph	59
14.7. Network	59
14.8. Optuna	59
14.9. Balance	59
14.10. Training	59
14.11. Evaluación	59
14.12. History	59
14.13. Experiments	60
15. Modelo de datos del grafo	60
15.1. Aristas y atributos	60
15.2. Etiquetado temporal y split purgado	60
15.3. Normalización y payload reproducible	61
16. Arquitectura del modelo GNN	61
16.1. Temporal head (opcional)	61
16.2. Registro de variantes GNN (encoder + temporal + aristas)	62
16.2.1. Idea general de comparabilidad	62
16.2.2. Variantes implementadas	62
16.2.3. Codificación de aristas (edge_attr)	63
16.2.4. Heads temporales y ensamblado de secuencias	63
16.2.5. Estrategias de cache temporal (punto crítico)	63
16.2.6. Matriz experimental recomendada	63
16.2.7. Métricas para eventos raros	64
16.2.8. Trazabilidad de variantes	64
17. Entrenamiento y optimización	64
17.1. Flujo del entrenamiento	64
17.2. Train-minibatch y regularización	65
17.3. Samplers de entrenamiento y RioGNN top-p RL	65
17.4. Early stopping y checkpoint BEST por pérdida de validación	66
18. GraphSMOTE: modificaciones sobre la arquitectura original	67
18.1. Cambio 1: de grafo homogéneo a HeteroData pórtico-minuto	67
18.2. Cambio 2: extractor heterogéneo y temporal en lugar de GraphSAGE fijo	67
18.3. Cambio 3: SMOTE sólo en entrenamiento y con embargo temporal	67

18.4. Cambio 4: decodificadores $z \rightarrow x$ para volver al espacio de features	68
18.5. Cambio 5: atributos de arista para aristas sintéticas	68
18.6. Cambio 6: conectividad top-K en lugar de adyacencia densa o umbral global	68
18.7. Cambio 7: modos offline y online sin acumulación de sintéticos	69
18.8. Cambio 8: pérdida, pesos de clase y diagnósticos	69
18.9. Cambio 9: trazabilidad y garantías de auditoría	69
19.ImGAGN: generación adversarial	69
19.1. Paso 1: homogenización	70
19.2. Paso 2: definir cantidad de sintéticos	70
19.3. Paso 3: generador	70
19.4. Paso 4: discriminador	70
19.5. Paso 5: entrenamiento adversarial	70
19.6. Paso 6: grafo final	71
20.Optuna (búsqueda de hiperparámetros)	71
21.Evaluación, calibración y artefactos	71
22.Auditoría técnica y mejores prácticas	72
22.1. Fortalezas observadas	72
22.2. Riesgos o puntos críticos (actualizado)	72
22.3. Recomendaciones priorizadas (estado actual)	73
23.Anexo: referencias a funciones y archivos	73
24.Implementación de MA-AIRL	74
24.1. Objetivo y alcance	74
24.2. Arquitectura general	74
24.3. Datos expertos y construcción del estado	74
24.4. Acciones expertas (proxy de SUMO)	74
24.5. Modelos de MA-AIRL	75
24.6. Función discriminadora y objetivo	75
24.7. Proxy temporal	75
24.8. Métricas de entrenamiento	75
24.9. Exportación de parámetros a SUMO	75
24.10 Integración en la UI	76
24.11 Archivos modificados	76
24.12 Limitaciones actuales y próximos pasos	76

1. Clustering

1.1. Objetivo

El objetivo del módulo de *clustering* es agrupar conductores (identificados por su matrícula) según indicadores de comportamiento observados en los registros de detección (**flujos**). Estos grupos se usan tanto para análisis exploratorio como para construir variables agregadas por pórtico/intervalo que alimentan modelos posteriores.

1.2. Datos y preprocesamiento

Partimos de un conjunto de observaciones $\{r_j\}_{j=1}^N$, donde cada registro contiene, al menos:

$$r_j = (t_j, p_j, \ell_j, v_j, \text{plate}_j),$$

con t tiempo, p p rtico, ℓ carril, v velocidad y plate la matr cula.

Normalizaci n de matr cula Se normaliza como texto en may sculas y se eliminan valores inv lidos (`,` `nan`, `null`, `none`). En la implementaci n se filtran matr culas con largo fuera de $[5, 6]$.

Limpieza de carril y duplicados El carril se convierte a num rico y se filtra a $\ell \in \{1, 2, 3\}$. Adem s, se eliminan duplicados exactos usando la llave:

$$(\text{plate}, p, \ell, t).$$

Tratamiento de at picos en velocidad Para cada grupo (p, ℓ) se calculan cuantiles q_α y q_β sobre la velocidad. Dependiendo del modo:

- **Winsorizaci n:** $v \leftarrow \min(\max(v, q_\alpha), q_\beta)$.
- **Filtrado:** se descartan observaciones con $v \notin [q_\alpha, q_\beta]$.
- **Sin acci n:** no se modifica v .

1.3. Ingenier a de *features* por conductor

Para cada conductor i (matr cula), se construye un vector de variables $x_i \in \mathbb{R}^d$ a partir de agregaciones sobre sus pasadas. En la implementaci n, estas variables se guardan en `Resultados/cluster_features*.duckdb` (tabla `cluster_features`).

1.3.1. Conteos, velocidad media y uso de carril

Sea N_i el n mero de pasadas del conductor i :

$$N_i = \sum_{j=1}^N \mathbf{1}[\text{plate}_j = i].$$

La velocidad media se define como:

$$\bar{v}_i = \frac{1}{N_i} \sum_{j: \text{plate}_j = i} v_j.$$

Para el uso de carril, definimos proporciones:

$$\pi_{i,k} = \frac{1}{N_i} \sum_{j: \text{plate}_j = i} \mathbf{1}[\ell_j = k], \quad k \in \{1, 2, 3\}.$$

1.3.2. Velocidad relativa

Se define una velocidad media de referencia por intervalo de 5 minutos y p rtico:

$$\bar{v}_{p,\tau} = \frac{1}{|\mathcal{I}_{p,\tau}|} \sum_{j \in \mathcal{I}_{p,\tau}} v_j, \quad \mathcal{I}_{p,\tau} = \{j : p_j = p, \lfloor t_j \rfloor_{5\min} = \tau\}.$$

Para cada pasada, la velocidad relativa es:

$$\rho_j = \frac{v_j}{\bar{v}_{p_j, \lfloor t_j \rfloor_{5\min}}},$$

y por conductor:

$$\bar{\rho}_i = \frac{1}{|\mathcal{J}_i|} \sum_{j \in \mathcal{J}_i} \rho_j, \quad \mathcal{J}_i = \{j : \text{plate}_j = i\}.$$

1.3.3. Headway (brecha temporal)

Para cada grupo (p, ℓ) (y opcionalmente por mes), se ordenan registros por (t, plate) . El *headway* de la observaci n j se calcula como:

$$h_j = t_j - t_{j-1},$$

en segundos, donde $j - 1$ es el registro anterior dentro del mismo grupo. Headways mayores que un umbral $h_{\max}$ (por defecto 60s) se tratan como faltantes. El *headway* medio por conductor:

$$\bar{h}_i = \frac{1}{|\mathcal{H}_i|} \sum_{j \in \mathcal{H}_i} h_j, \quad \mathcal{H}_i = \{j : \text{plate}_j = i, h_j > 0, h_j \leq h_{\max}\}.$$

1.3.4. TTC y tasa de conflictos

Sea $\Delta v_j = v_j - v_{j-1}$ la diferencia de velocidades entre el veh culo j y el anterior en el mismo (p, ℓ) . Para cierres ($\Delta v_j > 0$) se define el *Time-To-Collision*:

$$\text{TTC}_j = \frac{h_j v_j}{\Delta v_j}.$$

El umbral usado para marcar conflicto puede operar en dos modos:

$$\theta_j = \begin{cases} \tau_{p_j}, & \text{modo din mico,} \\ \tau_{\text{fixed}}, & \text{modo fijo.} \end{cases}$$

En el modo **din mico**, τ_{p_j} se obtiene del mapa por p rtico `TTC_MAX_BY_PORTICO`. En el modo **fijo**, τ_{fixed} aplica el mismo umbral a todos los p rticos; la interfaz ofrece valores de referencia de 1.0, 1.5, 2.0, 3.0 y 4.0 segundos, con 1.5 segundos como valor por defecto (`DEFAULT_FIXED_TTC_SECONDS`).

Se contabiliza un conflicto si $\text{TTC}_j < \theta_j$, y la tasa por conductor:

$$\text{conflict_rate}_i = \frac{\sum_{j \in \mathcal{C}_i} \mathbf{1}[\text{TTC}_j < \theta_j]}{|\mathcal{C}_i|}, \quad \mathcal{C}_i = \{j : \text{plate}_j = i, h_j > 0\}.$$

En la implementaci n, `Clusterization(..., ttc_mode="dynamic"|"fixed", fixed_ttc_s=...)` valida el modo seleccionado. El TTC solo se calcula cuando existe veh culo previo y el veh culo actual va m s r pido; para evitar valores extremos, TTC_j se acota por arriba a θ_j antes de computar la tasa. La configuraci n queda persistida mediante `build_ttc_feature_metadata(...)` en la tabla `cluster_features_meta` de los archivos `cluster_features*.duckdb`, con `ttc_mode`, `ttc_fixed_seconds` y `ttc_threshold_map`.

1.3.5. Cambios de carril

Ordenando las pasadas del conductor i por tiempo (y opcionalmente por mes), se define:

$$\text{lane_changes}_i = \sum_{m=2}^{N_i} \mathbf{1}[\ell_{i,m} \neq \ell_{i,m-1}], \quad \text{lane_change_rate}_i = \frac{\text{lane_changes}_i}{\text{máx}(N_i - 1, 1)}.$$

1.3.6. Ponderación mensual (opcional)

Cuando se activa ponderación mensual, se calculan variables por mes y luego se consolidan por matrícula mediante promedio ponderado por $N_{i,m}$ (pasadas del conductor i en el mes m):

$$\bar{x}_i = \frac{\sum_m N_{i,m} x_{i,m}}{\sum_m N_{i,m}}.$$

1.3.7. Actividad temporal

Para caracterizar cuán “persistente” es un conductor en el tiempo, se calculan métricas de actividad:

$$n_i^{(\text{days})} = |\{[t_j]_{\text{día}} : \text{plate}_j = i\}|, \quad n_i^{(\text{weeks})} = |\{[t_j]_{\text{semana}} : \text{plate}_j = i\}|, \quad n_i^{(\text{months})} = |\{[t_j]_{\text{mes}} : \text{plate}_j = i\}|,$$

Los meses se cuentan como meses calendario con año (por ejemplo, 2024-01 y 2025-01 son distintos). Estas columnas se materializan como `n_days_active`, `n_weeks_active`, `n_months_active` y `n_years_active`.

1.4. Preparación de datos para clustering

Se selecciona un subconjunto de columnas numéricas (*features*) y se descartan filas con valores faltantes o infinitos en dichas columnas. Antes de clusterizar se estandarizan las variables con *z-score*:

$$z = \frac{x - \mu}{\sigma},$$

usando `sklearn.preprocessing.StandardScaler`.

1.5. Métodos de clustering

1.5.1. K-means

K-means busca particionar los datos en K clusters minimizando la suma de distancias cuadráticas intra-cluster:

$$\min_{\{c_i\}, \{\mu_k\}} \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2.$$

En la implementación se usa `sklearn.cluster.KMeans` y, para evaluación rápida de K , opcionalmente `MiniBatchKMeans`.

Selección de K (métricas) Se calculan métricas para $K \in [K_{\text{mín}}, K_{\text{máx}}]$:

- **Silhouette:** $s(i) = \frac{b(i) - a(i)}{\text{máx}(a(i), b(i))}$.
- **Davies–Bouldin:** promedio de la razón entre dispersión intra-cluster y separación inter-cluster.
- **Calinski–Harabasz:** razón entre dispersión inter-cluster e intra-cluster.

1.5.2. Gaussian Mixture Models (GMM)

Un GMM modela la densidad como mezcla:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

estimada típicamente vía EM. La selección de K se apoya en criterios de información:

$$\text{AIC} = -2 \log L + 2p, \quad \text{BIC} = -2 \log L + p \log n,$$

donde L es la verosimilitud, p el número de parámetros y n el número de muestras. Además de la etiqueta dura $\arg \max_k p(z_i = k \mid x_i)$, GMM entrega probabilidades posteriores de pertenencia

$$r_{i,k} = p(z_i = k \mid x_i).$$

La interfaz puede exportar estas responsabilidades como columnas `cluster_prob_0`, `cluster_prob_1`, ..., para que etapas posteriores usen membresía suave en lugar de conteos duros.

1.5.3. HDBSCAN

HDBSCAN es un método basado en densidad que construye una jerarquía de clusters y extrae agrupaciones estables. Parámetros típicos:

- `min_cluster_size`: tamaño mínimo de cluster.
- `min_samples`: control de robustez ante ruido.

Las observaciones marcadas como ruido reciben etiqueta `-1`.

1.6. Conductores frecuentes vs. raros (opcional)

Para mejorar robustez cuando existen matrículas con pocas pasadas, el flujo de trabajo permite entrenar el modelo en *conductores frecuentes* (por ejemplo, $N_i \geq N_{\text{mín}}$) y luego asignar etiquetas a los raros con un umbral de confianza:

- **K-means**: se calcula la distancia mínima al centroide más cercano; si supera un percentil q estimado en el set frecuente, se etiqueta como desconocido (`-1`).
- **GMM**: se calcula la probabilidad posterior máxima; si es menor que un umbral γ , se etiqueta como desconocido (`-1`), pero las columnas `cluster_prob_*` pueden conservar la masa posterior completa.

Se guarda además un *confidence score* (`confidence_score`) y un indicador `is_rare`.

1.7. GMM con clusters dinámicos

El experimento **GMM/Clusters dinámicos** evalúa si la pertenencia de un conductor a los clusters aprendidos desde el historial se mantiene estable o cambia cuando sólo se observa su comportamiento reciente. Primero se entrena un GMM base sobre las variables históricas cargadas en la pestaña **Clustering**. Este entrenamiento fija el escalador $S(\cdot)$, los parámetros $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ y el conjunto de variables usado por el modelo.

Luego se define una ventana deslizante de x días con avance diario:

$$W_s = [d_s, d_s + x \text{ días}), \quad d_{s+1} = d_s + 1 \text{ día}.$$

Para cada ventana W_s , las variables por matrícula se recalculan directamente desde `flujos` usando la misma función `Clusterization`. No se reentrena el GMM dentro de la ventana; cada vector reciente $x_{i,s}$ se transforma con el escalador histórico y se asigna al GMM fijo:

$$r_{i,s,k} = p(z = k \mid S(x_{i,s})).$$

La etiqueta dura dinámica se define como:

$$\hat{c}_{i,s} = \begin{cases} \arg \max_k r_{i,s,k}, & \max_k r_{i,s,k} \geq \gamma \wedge N_{i,s} \geq N_{\min}, \\ -1, & \text{en otro caso,} \end{cases}$$

donde γ es el umbral de confianza y $N_{\min}$ el mínimo de pasadas requerido dentro de la ventana. Aunque una asignación quede como desconocida (-1) por baja confianza o bajo soporte, se conservan las probabilidades `cluster_prob_*` para analizar membresía suave.

La variante **GMM Dinámico prevalentes** mantiene exactamente el mismo entrenamiento histórico y el mismo GMM fijo, pero reduce la muestra usada en la asignación dinámica. Antes de procesar ventanas, se ordenan las patentes históricas por `n_months_active` descendente, `n_years_active` descendente, `total_passes` descendente y `plate` ascendente. La interfaz permite configurar un porcentaje q (por defecto 10 %) y selecciona exactamente

$$n_{\text{prev}} = \max \left(1, \left\lceil n_{\text{plates}} \frac{q}{100} \right\rceil \right)$$

patentes del historial cargado, sin expandir empates. Cada ventana filtra `flujos` a esas patentes antes de recalcular `Clusterization`; por tanto, el modelo base no cambia, sólo cambia la cobertura dinámica evaluada.

Para responder si la pertenencia varía o se mantiene, el módulo calcula métricas por matrícula:

$$\text{stability}_i = 1 - \frac{\sum_{s=2}^{m_i} \mathbf{1}[\hat{c}_{i,s} \neq \hat{c}_{i,s-1}]}{m_i - 1}, \quad \text{change_rate}_i = 1 - \text{stability}_i,$$

usando transiciones consecutivas con etiqueta asignada. También reporta el `dominant_cluster_share`, la confianza media y la entropía normalizada de las probabilidades:

$$H_{i,s} = -\frac{1}{\log K} \sum_{k=1}^K r_{i,s,k} \log r_{i,s,k}.$$

Los artefactos se guardan en `Resultados/dynamic_gmm_*.joblib` (modelo, escalador y metadatos) y `Resultados/dynamic_gmm_*.duckdb`, con tablas `dynamic_assignments`, `dynamic_window_summary` y `dynamic_metadata`. En el modo prevalente, los metadatos incluyen `assignment_scope`, `prevalent_fraction_pct`, `prevalent_plate_count`, `prevalent_source` y un hash estable de la selección para validar `resume/checkpoint`. La interfaz visualiza KPIs globales, composición temporal por cluster, matriz de transición y trayectoria individual por matrícula.

1.8. Salidas y artefactos

El módulo genera archivos en `Resultados/`:

- Variables por matrícula en DuckDB: `cluster_features*.duckdb`, incluyendo metadatos reproducibles de TTC en `cluster_features_meta`.
- Etiquetas por matrícula: `cluster_kmeans_kK.csv`, `cluster_gmm_kK.csv`, `cluster_hdbscan.csv`. En GMM, opcionalmente se agregan columnas `cluster_prob_*`.
- Resumen por cluster (*centroides*): `cluster_summary*.csv`.
- Estadísticos descriptivos por cluster: `cluster_descriptive*.csv`.

1.9. Variables de clusters agregadas por pórtico e intervalo

Además del clustering por matrícula, se construyen variables agregadas por pórtico/intervalo a partir de una tabla de etiquetas (`plate` $\rightarrow$ `cluster_label`). Para cada pórtico p , intervalo τ y cluster c , el modo duro usa:

$$n_{p,\tau,c} = \#\{j : p_j = p, \lfloor t_j \rfloor_{\Delta} = \tau, \text{cluster}(\text{plate}_j) = c\}, \quad \text{share}_{p,\tau,c} = \frac{n_{p,\tau,c}}{\sum_{c'} n_{p,\tau,c'}}.$$

Si el archivo contiene posteriores de GMM `cluster_prob_*`, la agregación usa membresía suave:

$$w_{p,\tau,c} = \sum_{j: p_j=p, \lfloor t_j \rfloor_{\Delta}=\tau} r_{j,c}, \quad \text{share}_{p,\tau,c} = \frac{w_{p,\tau,c}}{N_{p,\tau}},$$

donde $N_{p,\tau}$ es el número de detecciones con probabilidad de cluster disponible. En este modo, los conductores raros marcados como -1 siguen aportando según sus $r_{j,c}$, sin crear una columna adicional de “desconocido”. Opcionalmente se agregan:

- **Flow por hora:** $\text{flow}_{p,\tau,c} = m_{p,\tau,c} \cdot \frac{60}{\Delta} \cdot \frac{1}{L}$, donde $m = n$ en modo duro y $m = w$ en modo suave.
- **Velocidad media** por cluster: promedio simple en modo duro y promedio ponderado $\sum_j v_j r_{j,c} / \sum_j r_{j,c}$ en modo suave.
- **Densidad:** $\text{density}_{p,\tau,c} = \text{flow}_{p,\tau,c} / \bar{v}_{p,\tau,c}$ (con manejo defensivo cuando $\bar{v} \leq 0$).
- **Variaciones temporales:** Δspeed , $\Delta \text{density}$ vía diferencias entre intervalos consecutivos.
- **Entropía de mezcla de clusters:** $H_{p,\tau} = -\sum_c \text{share}_{p,\tau,c} \log(\text{share}_{p,\tau,c})$.

2. Predicción de accidentes

2.1. Objetivo

El objetivo es construir y evaluar modelos que predigan la ocurrencia de un accidente en un pórtico, utilizando variables agregadas a partir de detecciones vehiculares (**flujos**) y, opcionalmente, variables derivadas de *clusters* de conductores.

2.2. Fuentes de datos

Flujos (detecciones) Cada registro de detección contiene, al menos, un timestamp t , un pórtico p , una categoría vehicular c y una velocidad v . En el sistema, los flujos se almacenan en DuckDB y se consultan por rango temporal.

Eventos (accidentes) Los accidentes provienen de archivos de eventos. Se filtran por tipo *Accidente* y por *Vía* (por defecto, vía expresa). Luego se construye una marca temporal `accidente_time` y se mapea la localización (km, eje, calzada) a pórticos cercanos usando `Datos/Porticos.csv`. El pórtico asignado se guarda como `ultimo_portico`.

2.3. Discretización temporal y notación

Se discretiza el tiempo en intervalos de tamaño Δ minutos. Para una observación con timestamp t , definimos el inicio de intervalo como:

$$\tau(t) = \left\lfloor \frac{t}{\Delta} \right\rfloor \Delta.$$

Las variables se agregan por par (p, τ) , donde p es el pórtico.

2.4. Ingeniería de variables (features)

2.4.1. Variables macro de flujo

Para cada pórtico p , intervalo τ y categoría c , definimos:

$$n_{p,\tau,c} = \#\{\text{detecciones en } (p, \tau, c)\}, \quad \bar{v}_{p,\tau,c} = \frac{1}{n_{p,\tau,c}} \sum v.$$

Con L carriles para normalización, se calcula flujo por hora y densidad:

$$q_{p,\tau,c} = \frac{60}{\Delta} \frac{n_{p,\tau,c}}{L}, \quad k_{p,\tau,c} = \frac{q_{p,\tau,c}}{\bar{v}_{p,\tau,c}}.$$

Opcionalmente se agregan variables dinámicas vía primera diferencia temporal por pórtico:

$$\Delta \bar{v}_{p,\tau,c} = \bar{v}_{p,\tau,c} - \bar{v}_{p,\tau-\Delta,c}, \quad \Delta k_{p,\tau,c} = k_{p,\tau,c} - k_{p,\tau-\Delta,c}.$$

Finalmente se construye una matriz ancha (*wide*) con columnas del estilo `flow_<cat>`, `speed_<cat>`, `density_<cat>`, etc.

2.4.2. Variables de clusters agregadas (opcional)

Si existe un archivo de etiquetas por matrícula (`plate` $\rightarrow$ `cluster_label`), se unen las detecciones con su cluster correspondiente y se agregan por (p, τ, κ) , donde κ es el identificador de cluster. En modo de etiqueta dura:

$$n_{p,\tau,\kappa} = \#\{\text{detecciones en } (p, \tau, \kappa)\}, \quad \text{share}_{p,\tau,\kappa} = \frac{n_{p,\tau,\kappa}}{\sum_{\kappa'} n_{p,\tau,\kappa'}}.$$

Cuando el archivo proviene de GMM y contiene columnas `cluster_prob_*`, la ingeniería de variables usa membresía suave. Sea $r_{j,\kappa}$ la probabilidad posterior de la detección j para el cluster κ :

$$w_{p,\tau,\kappa} = \sum_{j \in (p,\tau)} r_{j,\kappa}, \quad \text{share}_{p,\tau,\kappa} = \frac{w_{p,\tau,\kappa}}{N_{p,\tau}}.$$

El flujo usa $w_{p,\tau,\kappa}$ como conteo esperado y la velocidad media se calcula como promedio ponderado por $r_{j,\kappa}$. Los conductores raros etiquetados como -1 siguen aportando a los clusters según sus probabilidades posteriores. Se derivan, opcionalmente, flujo, velocidad media, densidad y diferencias temporales por cluster, y una medida de mezcla:

$$H_{p,\tau} = - \sum_{\kappa} \text{share}_{p,\tau,\kappa} \log(\text{share}_{p,\tau,\kappa}).$$

Estas variables se almacenan como columnas `cluster_share_*`, `cluster_flow_*`, `cluster_speed_*`, etc.

2.5. Definición del target (etiqueta)

Sea un accidente con timestamp t_a y pórtico asignado p_a . El sistema define un *lead time* de un intervalo: el accidente se etiqueta en el intervalo anterior

$$\tau_a = \tau(t_a) - \Delta,$$

y el target se define como:

$$y_{p,\tau} = \begin{cases} 1, & \text{si existe un accidente con } (p_a, \tau_a) = (p, \tau), \\ 0, & \text{en caso contrario.} \end{cases}$$

Con esto, $y_{p,\tau} = 1$ representa “ocurrirá un accidente en el siguiente intervalo”.

2.6. Construcción del dataset

El dataset final contiene, por fila, un par (p, τ) con variables numéricas y **target**. La construcción es:

1. Calcular variables macro de flujo por (p, τ) .
2. (Opcional) Calcular variables de clusters por (p, τ) y unir las por llave (`portico`, `interval_start`).
3. Agregar **target** mediante unión con el conjunto de pares (p_a, τ_a) .

2.7. Partición temporal (train/validación/test)

Para evitar *leakage* temporal, se realiza un *split* por tiempo usando la columna `interval_start`: el conjunto de timestamps se ordena y se asigna el tramo final al test (proporción `test_size`). En entrenamiento estándar, además se separa validación desde el bloque de train/val con `val_size`.

2.8. Manejo de desbalance

La ocurrencia de accidentes es rara ($y = 1$). Se utilizan dos estrategias:

- **Ponderación de clases**: por ejemplo, Random Forest con `class_weight="balanced"`.
- **SMOTE (opcional)**: se genera un dataset balanceado aplicando SMOTE *solo sobre el conjunto de entrenamiento*. Las filas sintéticas se marcan con `synthetic=True` y no se usan para validación de umbral.

2.9. Modelos disponibles

La app entrena modelos sobre tres familias de variables: **Base** (solo flujo), **Cluster** (solo variables `cluster_*`) y **Base + Cluster** (ambas familias). Los modelos soportados en la pestaña **Modelos** son:

- **Random Forest**: ensamble de árboles; puede usar `class_weight` y `n_jobs`.
- **Balanced Random Forest**: variante de `imbalanced-learn` con submuestreo balanceado interno.
- **XGBoost**: gradiente boosting con objetivo binario logístico, regularización y `scale_pos_weight` automático opcional.
- **SVM**: clasificador con escalamiento (`StandardScaler`); los scores se orientan cuando es necesario antes de calibrar umbral.
- **Neural Network**: MLP tabular en PyTorch con `StandardScaler`, *early stopping*, `pos_weight` y ejecución en MPS/CUDA/CPU.

Cada modelo produce un *score* $s(x) \in \mathbb{R}$, que puede ser calibrado antes de seleccionar el umbral operativo.

2.10. Calibración de score y umbral operacional

Se define el *false alarm rate* (FAR) y la sensibilidad:

$$\text{FAR} = \frac{FP}{FP + TN}, \quad \text{Sens} = \frac{TP}{TP + FN}.$$

El helper `select_threshold_for_metric` selecciona θ sobre validación usando un objetivo explícito. Para FAR, con objetivo $\text{FAR} \leq \alpha$, se elige:

$$\theta = \arg \max_{\theta: \text{FAR}(\theta) \leq \alpha} \text{Sens}(\theta),$$

y luego se evalúa en test con $\hat{y} = \mathbf{1}[s(x) \geq \theta]$.

Además de FAR, la app permite escoger umbrales por **F1**, **Balanced F1**, **MCC**, **Recall@N** **alertas/día** y **Costo operacional**. Los objetivos **PR-AUC** y **ROC-AUC** son métricas de ranking; cuando se usan como criterio de threshold, el umbral operativo queda en 0.5. La calibración de score puede ser `none`, `sigmoid` (Platt scaling) o `isotonic`.

Los protocolos de evaluación son:

- **Conservador**: entrena con `train`, calibra score y umbral en `validation`, y evalúa una vez en `test`.
- **Robusto**: combina `train+validation`, genera predicciones OOF con folds temporales, calibra score y umbral en OOF, reentrena con `train+validation` y evalúa en `test`.

2.11. Métricas de evaluación

En test se reportan, entre otras:

Accuracy, Precision, Recall, *F1*, FAR, Sens, ROC-AUC, PR-AUC,

además de **Balanced F1**, **MCC**, **Brier score**, alertas por día, costo operacional y matriz de confusión.

2.12. Pestaña Optuna

La pestaña **Optuna** usa la selección activa de variables y crea configuraciones separadas:

- **Base**: variables numéricas seleccionadas que no comienzan con `cluster_`.
- **Cluster**: variables seleccionadas de la familia `cluster_*`.
- **Base + Cluster**: todas las variables numéricas seleccionadas.

Para cada configuración se ejecutan variantes **sin balance interno** y, cuando `imbalanced-learn` está disponible, **con SMOTE**. La optimización usa `TPESampler`, poda opcional con `MedianPruner`, `n_jobs` para trials paralelos y controles separados para paralelismo interno de Random Forest o XGBoost.

Optuna puede ajustar hiperparámetros de Random Forest, XGBoost, SVM y Neural Network; también puede optimizar `top_k`, usando un ranking calculado sobre train real por Random Forest o mutual information. La métrica objetivo de Optuna se selecciona de un catálogo amplio (`f1`, `roc_auc`, `pr_auc`, `balanced_f1`, `mcc`, `brier_score`, `recall_at_alerts_per_day`, `operational_cost`, entre otras) y queda separada del objetivo usado para elegir `threshold`.

Los resultados se guardan en `Resultados/optuna_*.json` y `Resultados/optuna_*trials.csv`. El JSON conserva `dataset_fingerprint`, selección de variables, `feature_cols`, `search_space`, `optuna_settings` y resultados agrupados por modelo, `by_balance_mode` y `by_calibration_method`. La lectura canónica en memoria es `optuna_results_store`; `_get_active_optuna_best()` prefiere la variante SMOTE cuando existe y respeta la calibración seleccionada.

2.13. Pestaña Modelos

La pestaña **Modelos** entrena Base, Cluster y Base + Cluster con parámetros independientes. Antes de entrenar, `_apply_optuna_model_params_to_state(...)` intenta aplicar los mejores hiperparámetros de Optuna a cada grupo de variables; el match exige mismo modelo, modo de balance y calibración, salvo que el usuario habilite el fallback de calibración.

El manejo de desbalance puede ser:

- **Sin balance interno**.
- **Class weight / scale_pos_weight**: usa pesos de clase o `scale_pos_weight` automático según el modelo.
- **SMOTE interno**: aplica SMOTE solo dentro de train/fold, nunca en validación ni test.

Si se usa un dataset balanceado legacy ya generado en la pestaña **Balance**, la corrida evita aplicar doble SMOTE sobre el mismo split. Al terminar, la pestaña muestra métricas por protocolo, deltas contra Base, matrices de confusión y guarda la corrida en `Resultados/experiment_history.jsonl` y `Resultados/model_history/`.

2.14. Pestaña Experimentos

La pestaña **Experimentos** tiene tres vistas: **Ejecutar Nuevo**, **Resultados Anteriores** e **Importar Experimento**. Los experimentos nuevos disponibles son:

- **Features sampler**: barre top- K variables y compara Base contra Base + Cluster mediante `ExperimentsRunner.run_iterative_experiment`.

- **Find samples sizes:** explora tamaños/rangos de muestra para encontrar configuraciones con suficientes positivos.
- **Best highway section:** barre tramos de autopista con el protocolo controlado y registra el mejor segmento.
- **Best mix Highway section & K:** combina búsqueda de tramo y grilla de K .
- **Calibración score + threshold:** barre métrica objetivo de Optuna, calibración del score, objetivo operacional de threshold y modo de balance; usa protocolo robusto fijo y persiste checkpoints en `Resultados/calibration_experiment_runs/`.
- **Comparación controlada:** compara modelos, sets de variables, modos de balance, protocolos de threshold y grilla de K sobre un split temporal congelado.

Los resultados anteriores detectan automáticamente el tipo de experimento, permiten visualizar tablas/gráficos y exportar o importar paquetes ZIP con archivos relacionados.

2.15. Artefactos

El flujo genera artefactos en `Resultados/` (nombres con timestamp/sufijo):

- Features de flujo: `accident_flow_features_*.csv` y/o `.duckdb`.
- Features de clusters: `accident_cluster_features_*.csv`.
- Datasets balanceados por familia: `accident_balanced*_base.csv`, `accident_balanced*_cluster_only.csv` y `accident_balanced*_base_cluster.csv`.
- Resultados Optuna: `optuna_*.json` y `optuna_*trials.csv`.
- Resultados de experimentos: `experiments_results_*.csv`, `controlled_comparison_summary_*.csv`, `controlled_comparison_curves_*.csv`, `controlled_comparison_grid_*.csv`, `best_highway_section_con` y carpetas de checkpoint en `calibration_experiment_runs/`.
- Bases live de experimentos: `experiment_live_*.sqlite`, con tablas `meta`, `results` y `best`.
- Bundles persistentes de modelos: `model_history/<run_id>/<strategy>/model.joblib`, `manifest.json`, `background.parquet` y `explain_rows.parquet`.

2.16. XAI para Base+Cluster

Al finalizar el entrenamiento en la pestaña **Modelos**, la estrategia **Base+Cluster** persiste un bundle reproducible bajo `Resultados/model_history/`. El `manifest.json` registra el modelo, hiperparámetros, columnas usadas, métricas, umbral calibrado, split final, procedencia de las *flow features*, procedencia exacta de las variables de cluster, selección de variables y si se usó balanceo. Con esto, el historial puede reabrir la explicación sin recalcular features ni recargar manualmente archivos de clusters.

La explicabilidad se calcula con SHAP sobre dos subconjuntos persistidos:

- **Background:** hasta 128 filas reales de entrenamiento para construir el baseline del explainer.
- **Explain rows:** hasta 64 filas de test con `target`, `score`, `pred` y `threshold`, enriquecidas con casos representativos como mayor score, TP fuerte, FP fuerte y FN relevante.

La vista XAI reporta:

- Un ranking global por $\text{mean}(|\text{SHAP}|)$ para todas las variables del modelo **Base+Cluster**.
- Una agregación por familia de variables **Base** vs **Cluster**, para cuantificar qué proporción de la contribución explicativa total proviene de las variables `cluster_*`.
- Un top específico de variables de cluster.
- Casos locales representativos, mostrando qué variables empujan el riesgo hacia arriba o hacia abajo para cada observación.

Para modelos basados en árboles (**Random Forest** y **XGBoost**) se usa **TreeExplainer**; para **SVM** se usa **KernelExplainer** sobre la salida probabilística positiva. Si SHAP falla o la corrida es antigua y no tiene bundle, la interfaz informa que XAI no está disponible, pero mantiene el resto del historial operativo.

3. Ingeniería de *features* con GMM dinámico y DuckDB

3.1. Propósito y alcance

Esta sección documenta el cálculo de variables de flujo y de clusters para predicción de accidentes cuando la fuente de detecciones es masiva y se utiliza un GMM dinámico. El objetivo metodológico es preservar la semántica estadística de las variables originales, pero cambiar el plan de ejecución para que DuckDB procese los registros crudos por lote y pandas reciba únicamente agregados por pórtilo e intervalo.

La corrección es necesaria porque la base `Datos/flujos.duckdb` no puede tratarse como una tabla pequeña. En la inspección local de esquema, la tabla productiva `flujos_duckdb` contiene:

- **Tamaño físico:** aproximadamente 39 GB.
- **Filas:** 3,666,832,634 detecciones.
- **Rango temporal:** desde 2018-01-01 00:00:00 hasta 2024-09-30 23:59:59.
- **Columnas:** `FECHA` `TIMESTAMP`, `VELOCIDAD` `DOUBLE`, `CATEGORIA` `INTEGER`, `MATRICULA` `VARCHAR`, `PORTICO` `VARCHAR` y `CARRIL` `VARCHAR`.

Por lo tanto, cualquier implementación correcta debe empezar consultando el esquema real de las bases, usar filtros temporales y espaciales antes de materializar resultados, y evitar `SELECT *` sobre tablas crudas. El cálculo actual está diseñado bajo esa restricción.

3.2. Contrato de datos

3.2.1. Base de flujos

La fuente canónica de detecciones es `Datos/flujos.duckdb`, tabla `flujos_duckdb`. Cada fila representa una detección vehicular:

$$x_j = (t_j, p_j, m_j, c_j, v_j, \ell_j),$$

donde t_j es `FECHA`, p_j es `PORTICO`, m_j es `MATRICULA`, c_j es `CATEGORIA`, v_j es `VELOCIDAD` y ℓ_j es `CARRIL`. Para el cálculo de features de accidente se usan sólo las columnas necesarias: `FECHA`, `PORTICO`, `MATRICULA`, `VELOCIDAD` y `CATEGORIA`; `CARRIL` no se requiere en esta etapa.

La discretización temporal usa intervalos de $\Delta = 5$ minutos en la interfaz actual:

$$\tau_j = \text{bucket}_\Delta(t_j).$$

En DuckDB se implementa mediante `time_bucket(INTERVAL '5 minutes', FECHA)`.

3.2.2. GMM dinámico

El GMM dinámico se consume desde un archivo DuckDB externo con tabla `dynamic_assignments`. Su contrato mínimo es:

- **plate**: patente o matrícula.
- **window_end**: timestamp que indica hasta cuándo se observó la ventana dinámica usada para asignar la patente.
- **cluster_label**: etiqueta dura del cluster dominante.
- **cluster_prob_0**, **cluster_prob_1**, ...: probabilidades posteriores de pertenencia al GMM, cuando están disponibles.

Opcionalmente existe `dynamic_metadata`, con pares `key`, `value_json`, para registrar configuración del experimento, alcance de asignaciones y metadatos reproducibles.

3.3. Variables base de flujo

3.3.1. Mapeo de categorías

La categoría vehicular cruda se normaliza a tres familias:

$$g(c) = \begin{cases} \text{Light}, & c = 1, \\ \text{Heavy}, & c \in \{2, 3\}, \\ \text{Motorcycles}, & c = 4, \\ \emptyset, & \text{otro caso.} \end{cases}$$

Las categorías no reconocidas se excluyen del agregado. Las columnas anchas se generan por combinación de métrica y categoría, por ejemplo `flow_light`, `speed_heavy`, `density_motorcycles`.

3.3.2. Agregación por pórtico e intervalo

Para pórtico p , intervalo τ y categoría a , se define el conjunto:

$$\mathcal{I}_{p,\tau,a} = \{j : p_j = p, \tau_j = \tau, g(c_j) = a\}.$$

El conteo de detecciones es:

$$n_{p,\tau,a} = |\mathcal{I}_{p,\tau,a}|.$$

La velocidad media y desviación estándar muestral son:

$$\bar{v}_{p,\tau,a} = \frac{1}{n_{p,\tau,a}} \sum_{j \in \mathcal{I}_{p,\tau,a}} v_j, \quad s_{p,\tau,a} = \text{sd}\{v_j : j \in \mathcal{I}_{p,\tau,a}\}.$$

Cuando sólo existe una observación, la desviación estándar se define como cero para mantener columnas numéricas sin valores indefinidos.

Con L carriles de normalización y Δ minutos por intervalo, el flujo horario por carril es:

$$q_{p,\tau,a} = \frac{60}{\Delta} \frac{n_{p,\tau,a}}{L}.$$

La densidad se aproxima con la relación macroscópica $k = q/v$:

$$k_{p,\tau,a} = \begin{cases} q_{p,\tau,a}/\bar{v}_{p,\tau,a}, & \bar{v}_{p,\tau,a} > 0, \\ 0, & \text{en otro caso.} \end{cases}$$

3.3.3. Deltas temporales

Las diferencias temporales se calculan por pórtico y categoría:

$$\Delta \bar{v}_{p,\tau,a} = \bar{v}_{p,\tau,a} - \bar{v}_{p,\tau-\Delta,a}, \quad \Delta k_{p,\tau,a} = k_{p,\tau,a} - k_{p,\tau-\Delta,a}.$$

La implementación DuckDB usa `LAG(...)` particionado por `portico` y `category_label`. En modo batch, estos deltas son **locales al lote**: el primer intervalo disponible dentro de una semana o mes recibe delta cero si el intervalo anterior quedó fuera del lote. Esto preserva el comportamiento operativo previo, que también procesaba por lotes.

3.4. Variables de cluster: etiqueta dura versus membresía probabilística

3.4.1. Modo de asignación dura

Cuando sólo se dispone de `cluster_label`, cada detección con patente emparejada aporta masa completa a un único cluster:

$$z_{j,k} = \mathbf{1}\{y_j = k\},$$

donde y_j es la etiqueta asignada a la patente de la detección. Para p, τ, k :

$$n_{p,\tau,k} = \sum_{j \in (p,\tau)} z_{j,k}, \quad N_{p,\tau} = \sum_k n_{p,\tau,k}.$$

La proporción de cluster es:

$$\text{share}_{p,\tau,k} = \frac{n_{p,\tau,k}}{N_{p,\tau}}.$$

En este modo, si existe la etiqueta -1 , se materializa como columna `cluster_share_neg_1`, `cluster_flow_neg_1`, etc. Esto representa explícitamente el grupo desconocido o de baja confianza.

3.4.2. Modo de membresía suave con probabilidades GMM

Cuando existen columnas `cluster_prob_*`, el cálculo no debe reducir el GMM a $\arg \max$ si el objetivo es conservar incertidumbre. Sea:

$$r_{j,k} = P(Z_j = k \mid \mathbf{x}_{m_j, s(j)}),$$

donde $\mathbf{x}_{m_j, s(j)}$ es el vector de comportamiento reciente de la patente m_j en la ventana dinámica causal seleccionada. En este caso:

$$z_{j,k} = r_{j,k}, \quad 0 \leq r_{j,k} \leq 1, \quad \sum_k r_{j,k} \approx 1.$$

El conteo por cluster se interpreta como conteo esperado:

$$w_{p,\tau,k} = \sum_{j \in (p,\tau)} r_{j,k}.$$

La proporción queda:

$$\text{share}_{p,\tau,k} = \frac{w_{p,\tau,k}}{N_{p,\tau}},$$

donde $N_{p,\tau}$ es el número de detecciones con asignación dinámica causal. El denominador no es $\sum_k w_{p,\tau,k}$ por accidente: bajo normalización posterior ambos coinciden, pero usar $N_{p,\tau}$ hace explícito que cada detección aporta una unidad de masa repartida entre clusters.

La diferencia fundamental es epistemológica y operacional:

- **Etiqueta dura:** impone una decisión discreta y destruye la incertidumbre posterior. Es apropiada cuando el modelo aguas abajo requiere particiones mutuamente excluyentes o cuando las probabilidades no están disponibles.
- **Membresía suave:** propaga la incertidumbre del GMM hacia las variables macroscópicas. Un vehículo con probabilidades (0,55, 0,45) no es tratado igual que uno con (0,99, 0,01), aunque ambos tengan el mismo $\arg \max$.
- **Etiqueta -1:** en modo duro genera una familia de columnas `neg_1`. En modo suave, una patente etiquetada como -1 puede seguir aportando a los clusters k mediante sus probabilidades `cluster_prob_*`; no se crea una columna `neg_1` salvo que no existan probabilidades.

3.5. Métricas agregadas por cluster

Para cada pórtico p , intervalo τ y cluster k , la implementación produce, según las opciones seleccionadas:

$$\begin{aligned} \text{cluster_share_}k &: \text{share}_{p,\tau,k}, \\ \text{cluster_flow_}k &: \frac{60}{\Delta} \frac{w_{p,\tau,k}}{L}, \\ \text{cluster_speed_}k &: \frac{\sum_{j \in (p,\tau)} v_j z_{j,k}}{\sum_{j \in (p,\tau)} z_{j,k}}, \\ \text{cluster_density_}k &: \frac{\text{cluster_flow_}k}{\text{cluster_speed_}k}. \end{aligned}$$

En modo duro, $z_{j,k}$ es indicador y $w = n$. En modo suave, $z_{j,k} = r_{j,k}$.

Las diferencias temporales por cluster son:

$$\Delta \text{cluster_speed}_{p,\tau,k} = \text{cluster_speed}_{p,\tau,k} - \text{cluster_speed}_{p,\tau-\Delta,k},$$

$$\Delta \text{cluster_density}_{p,\tau,k} = \text{cluster_density}_{p,\tau,k} - \text{cluster_density}_{p,\tau-\Delta,k}.$$

También se calcula la entropía de mezcla:

$$H_{p,\tau} = - \sum_k \text{share}_{p,\tau,k} \log(\text{share}_{p,\tau,k}),$$

con la convención $0 \log 0 = 0$. Valores bajos indican dominio de un cluster; valores altos indican mezcla heterogénea de patrones de conducción.

3.6. Alineamiento causal del GMM dinámico

3.6.1. Filtración temporal

El GMM dinámico entrega varias asignaciones para una misma patente a través del tiempo. Para evitar *look-ahead bias*, una detección en tiempo t_j sólo puede usar una asignación cuya ventana terminó antes o en t_j . Formalmente, para patente m y asignaciones con fin de ventana $u_{m,s}$:

$$s^*(j) = \arg \max_s \{u_{m,s} : u_{m,s} \leq t_j\}.$$

Si no existe tal $s^*(j)$, la detección no tiene asignación dinámica previa y no aporta variables de cluster. En el dataset final, al unir con las features base, las columnas de cluster quedan en cero para esa llave (p, τ) .

3.6.2. Implementación con ASOF JOIN

La operación causal se implementa dentro de DuckDB con:

```
ASOF LEFT JOIN dynamic_assignments sobre plate_clean y FECHA >= window_end.
```

El join es causal porque selecciona la asignación más reciente no futura para la patente. Además:

- Las patentes se normalizan a mayúsculas y sin espacios.
- Se descartan valores vacíos, NULL, NaN y equivalentes textuales.
- Para cada lote se filtra `dynamic_assignments` por patentes presentes en ese lote, evitando escanear o materializar asignaciones irrelevantes.
- Se exige `window_end < batch_end`; luego el `ASOF JOIN` refina a `window_end <= FECHA`.

Esta combinación evita usar información futura y reduce memoria, porque DuckDB ejecuta el emparejamiento cerca de los datos.

3.7. Plan de ejecución por lotes

La ejecución actual en `src/cluster_accident_app.py` sigue este flujo:

1. Crear el archivo de salida `Resultados/accident_flow_features*.duckdb`.
2. Adjuntar `Datos/flujos.duckdb` como `flow_db` en modo `READ_ONLY`.
3. Si se usa GMM dinámico, adjuntar el DuckDB de GMM como `dynamic_gmm_db` en modo `READ_ONLY`.
4. Consultar `mín(FECHA)` y `máx(FECHA)` con filtros de fecha y tramo para construir lotes semanales o mensuales.
5. Para cada lote, ejecutar `_compute_flow_features_duckdb_batch(...)` sobre columnas mínimas y filtros `FECHA`, `PORTICO`.
6. Si el modo es GMM dinámico, ejecutar `_compute_dynamic_gmm_cluster_features_duckdb_batch(...)` sin cargar `dynamic_assignments` a pandas.

7. Unir en pandas sólo los agregados (p, τ) con las variables de segmentos `last_/next_`. Esta etapa opera sobre miles o millones de filas agregadas, no sobre miles de millones de detecciones crudas.
8. Insertar cada `final_batch` en la tabla `flow_features` del DuckDB de salida.

La ruta estática de clusters CSV puede seguir usando pandas por lote, porque el archivo de etiquetas por patente es mucho menor que `flujos.duckdb`. La ruta dinámica, en cambio, debe permanecer en DuckDB para no materializar toda la tabla `dynamic_assignments`.

3.8. Modelo de memoria y política de carga

La regla operativa es: **pandas nunca debe recibir tablas crudas completas**. En particular:

- No se crea `work_flujos AS SELECT *` para todo el rango.
- No se hace `SELECT *` de `flujos_duckdb` hacia pandas.
- No se carga `dynamic_assignments` completa durante el cálculo Streamlit con GMM dinámico.
- No se carga automáticamente `flow_features` completa al terminar si el usuario no lo solicita.

El resultado se guarda siempre en DuckDB. La carga completa a memoria al finalizar queda controlada por el checkbox **Cargar resultado completo a memoria al finalizar**. Incluso si el usuario lo activa, la carga se bloquea cuando el resultado supera `FEATURE_AUTOLOAD_ROW_LIMIT = 250000` filas. En ese caso se guarda el path, se muestra un preview y se solicita carga filtrada desde **Cargar existentes**. La carga de archivos existentes aplica el mismo principio: primero perfila con `COUNT(*)`; si la consulta supera el límite, no se trae completa salvo confirmación explícita.

3.9. Salida y compatibilidad

La salida canónica es un DuckDB con tabla `flow_features`. Cada fila representa un segmento entre pórticos y un intervalo temporal:

$$(p_{\text{last}}, p_{\text{next}}, \tau).$$

Las variables del pórtico aguas arriba llevan prefijo `last_`; las del pórtico aguas abajo llevan prefijo `next_`. Por ejemplo:

- `last_flow_light, next_flow_light`.
- `last_cluster_share_0, next_cluster_share_0`.
- `last_cluster_entropy, next_cluster_entropy`.

El helper `_load_dynamic_gmm_assignments(...)` permanece disponible para pruebas y compatibilidad, pero no forma parte del cálculo Streamlit optimizado. La ruta productiva con GMM dinámico usa DuckDB directamente mediante `_compute_dynamic_gmm_cluster_features_duckdb_batch(...)`.

3.10. Validación y pruebas

La implementación se valida con pruebas unitarias focalizadas:

- Equivalencia entre el helper SQL de GMM dinámico y la implementación pandas para membresía suave.
- Equivalencia para etiquetas duras cuando no existen columnas `cluster_prob_*`.
- Caso sin asignación causal previa: las filas no contaminan clusters futuros y se preservan columnas esperadas.
- Equivalencia del cálculo base DuckDB contra `compute_flow_features` en un dataset pequeño.
- Política de carga: resultados pequeños pueden cargarse a memoria; resultados sobre el límite quedan como path y preview.

Los comandos de regresión recomendados son:

```
.venv/bin/python -m pytest tests/test_cluster_accident_dynamic_gmm_features.py -q  
.venv/bin/python -m pytest tests/test_crash_prediction_feature_engineering.py -q
```

3.11. Implicaciones científicas

Desde el punto de vista de modelamiento, la membresía suave transforma las variables de cluster desde conteos categóricos a estimadores de esperanza condicional. Esto reduce la varianza inducida por decisiones arg máx inestables y preserva información sobre incertidumbre individual. La etiqueta dura es una aproximación degenerada del caso probabilístico, equivalente a reemplazar el vector posterior por un vector one-hot.

El alineamiento causal es indispensable para predicción de accidentes: si una detección usa una asignación estimada con una ventana que termina después de la detección, el modelo recibiría información del futuro. La condición `window_end ≤ FECHA`, implementada por `ASOF JOIN`, convierte la tabla dinámica en un proceso adaptado a la filtración temporal disponible en cada instante.

Finalmente, la decisión de ejecutar agregaciones en DuckDB no es sólo una optimización de ingeniería. También mejora reproducibilidad: el contrato de entrada queda explícito, las consultas son determinísticas por lote, el resultado queda persistido como `flow_features`, y el historial registra path, perfil de filas/columnas, rango temporal, parámetros y artefactos de entrada.

4. Réplica del paper en NLP in Severity

4.1. Propósito y alcance

El pipeline de réplica del paper implementado en `src/nlp_severity_app.py` no es el mismo flujo de predicción de ocurrencia de accidentes documentado en la Sección 2. Su objetivo es reproducir el estudio de ablación reportado en `NLP/Latex/main.tex`, donde:

- **M1** usa solo variables de flujo,
- **M2** usa solo embeddings narrativos,
- **M3** usa la representación multimodal fusionada.

El punto de entrada es `run_paper_replication`, expuesto en la subpestaña `Paper replication` de la app Streamlit `NLP in Severity`. El código agrega, además, controles de reproducibilidad que no aparecen en el manuscrito: ejecución en tres rutas (`frozen`, `raw` y `update_embeddings`), reanudación/carga por checkpoints, comparación estricta entre rutas y promoción condicionada de assets hacia LaTeX.

4.2. Protocolo bloqueado en código

El protocolo de réplica está definido por constantes como `PAPER_PROTOCOL`, `PAPER_EXPECTED_COUNTS`, `PAPER_K_GRID`, `PAPER_VALIDATION_ALPHA` y `PAPER_COMPARISON_TOLERANCE`. La implementación fuerza:

- Target binario `severity_target`.
- Holdout temporal con `test_size = 0.2` y `random_state = 42`. El código ordena por `accidente_time` y asigna el último 20 % al test. Si la columna temporal no existe o produce splits con una sola clase, el código usa split estratificado aleatorio como fallback.
- XGBoost como única familia de modelos en la réplica del paper.
- Conteos esperados del dataset congelado: 2.070 filas, 432 variables de flujo, 200 variables de embeddings, 632 variables totales, con split 1.656/414, clases de entrenamiento 425 No-MARC / 1.231 MARC y clases de test 44 No-MARC / 370 MARC.
- Números candidatos de variables en modo `manual` (por defecto):

$$k \in \{10, 15, 20, 25, 30, 40, 50, 70, 100, 150, 200, 250, 300, 350, 400, 432\},$$

recortados al número de variables disponibles en cada modalidad. El usuario selecciona un subconjunto de esta grilla. Un modo alternativo `interval_max` genera $k = \{\Delta, 2\Delta, \dots, k_{\text{máx}}\}$ con paso configurable $\Delta \in \{5, 10, 25, 50, 100\}$.

- Métrica de scoring configurable para la búsqueda CV interna. Valores soportados: `f1` (por defecto), `roc_auc`, `recall`, `precision`, `accuracy`.
- Puntaje de validación

$$\text{Score}_{\text{val}} = \alpha \cdot F1_{\text{global}} + (1 - \alpha) \cdot (1 - FNRate),$$

con $\alpha = 0,3$ y tolerancia de comparación entre rutas $\varepsilon = 0,001$.

- Folds de CV configurables (2–10, por defecto 5) tanto para la validación cruzada interna como externa.

4.3. Ejecución paso a paso

4.3.1. Etapa 1: Construcción del contexto de ejecución

`run_paper_replication` comienza llamando a `_paper_build_execution_context`. Esta etapa reúne los insumos que definen la reproducibilidad:

1. fingerprint del dataset congelado `NLP/Dataframes/resultado.pkl`,
2. fingerprint de la fuente `raw`, que puede ser:

- el `accidents_df` en memoria que llega desde la UI, o
 - el último artefacto `processed_events` del registry,
3. fingerprint del modelo Transformer fine-tuneado resuelto desde el registry,
 4. snapshot del protocolo bloqueado del paper.

Esos fingerprints se convierten en un hash determinístico llamado `computed_run_id`. Esa clave es la que se usa para decidir si un checkpoint anterior es compatible.

4.3.2. Etapa 2: Reanudar, cargar o comenzar limpio

Antes de ejecutar cualquier experimento, el código revisa `Resultados/nlp_in_severity/paper_replication/` buscando un checkpoint compatible:

- si el checkpoint está `completed`, el payload se carga desde disco y no se reentrena ningún modelo;
- si el checkpoint está `running` o `failed`, el pipeline retoma desde el primer paso durable incompleto;
- si no existe checkpoint compatible, se crea un manifest nuevo.

El directorio del run contiene `manifest.json`, `live_status.json`, `live_events.jsonl` y los subdirectorios `frozen/`, `raw_build/`, `raw/`, `update_emb_build/`, `update_emb/`, `compare/` y `export/`.

4.3.3. Etapa 3: Ruta congelada

La ruta `frozen` carga NLP/Dataframes/`resultado.pkl` y normaliza el dataset con `_ensure_paper_dataset_col`. Luego `_paper_dataset_validation_report` verifica:

- número de filas,
- cantidad de variables de flujo y embeddings,
- tamaños de train/test,
- distribución de clases en train/test,
- si esos valores coinciden con las expectativas bloqueadas del paper.

Esta validación se registra como el paso durable `frozen.dataset_validation`. Es la ruta base y la referencia que luego se usa para comparar la reconstrucción `raw`.

4.3.4. Etapa 4: Ruta de reconstrucción raw

La ruta `raw` se construye en `_paper_build_raw_dataset`. Reconstruye el dataset del paper desde eventos fuente y registros AVI:

1. **Resolución de fuente.** Usar `accidents_df` desde la UI si está disponible; de lo contrario, cargar el último artefacto `processed_events`.
2. **Ingeniería de variables de flujo.** Llamar a `build_severity_feature_dataset` con cinco ventanas de un minuto antes y después de cada choque, las cuatro métricas granulares (`flow`, `speed_mean`, `speed_std`, `density`) y deltas temporales activados.

3. **Filtrado por cobertura.** Excluir accidentes que no tienen cobertura de flujo en los pórticos AVI aguas arriba/abajo.
4. **Construcción de texto.** Formar `text_bert` a partir de `accidente_time`, `km`, `eje`, `calzada`, `subtipo` y `descripcion`.
5. **Resolución del Transformer fine-tuneado.** Cargar el modelo de clasificación más reciente registrado para `text_bert`.
6. **Extracción de embeddings.** Llamar a `generate_text_embeddings` con `method = transformer_finetuned`, `max_length = 128`, `batch_size = 16` y `transformer_projection = cls`. El código usa el estado oculto final del primer token mediante `_project_transformer_hidden_state`.
7. **Reducción supervisada del embedding.** Rankear las dimensiones del embedding con Random Forest y conservar las 200 mejores con `_select_top_embedding_features`.
8. **Ensamble final del dataset.** Unir las 432 variables de flujo con las 200 dimensiones narrativas seleccionadas para producir el dataset de 632 variables alineado con el paper.

Estos subpasos quedan checkpointed como `raw.build.features`, `raw.build.embeddings`, `raw.build.embedding_selection` y `raw.build.dataset`.

4.3.5. Etapa 5: Corridas por modalidad (M1, M2, M3)

Cada ruta llama a `_paper_run_route`, que ejecuta hasta tres experimentos restringidos por modalidad (el usuario puede seleccionar un subconjunto):

- **M1:** Solo flujo,
- **M2:** Solo embeddings,
- **M3:** Todo.

Para cada modelo, `_paper_build_model_result` realiza:

1. resolución del grupo de variables con `_resolve_feature_group`,
2. construcción del holdout temporal mediante `_prepare_holdout_split_with_ids` (fallback estratificado),
3. imputación por mediana en el split de entrenamiento,
4. ranking de variables con Random Forest sobre los datos imputados pre-SMOTE (para evitar filtración por muestras sintéticas),
5. búsqueda sobre la grilla de k usando validación cruzada anidada,
6. selección de k^* por mejor `validation_score` (puntaje más alto gana; el k más pequeño desempata),
7. ajuste final de XGBoost y búsqueda de hiperparámetros sobre todo el split de entrenamiento, evaluación única sobre el test fijo.

Cada k se registra por separado como `<ruta>.M<j>.k.<k>`, y cada modelo final como `<ruta>.M<j>.final`.

4.3.6. Etapa 6: Nested CV y ajuste final

La evaluación anidada implementada en `_paper_nested_xgb_validation` utiliza:

- folds estratificados externos configurables (por defecto 5, rango 2–10),
- folds estratificados internos configurables (por defecto 5) cuando la frecuencia de clases lo permite,
- imputación por mediana dentro de cada fold externo,
- el balanceo con SMOTE se aplica dentro de la búsqueda interna de hiperparámetros (`_paper_optimize_xgb_model`) no directamente sobre el fold externo de entrenamiento. La búsqueda interna optimiza conjuntamente los hiperparámetros de XGBoost y los parámetros de SMOTE (`sampling_strategy`, `k_neighbors`).

La grilla de hiperparámetros de XGBoost depende del perfil de tuning seleccionado:

- **Rapida** (por defecto):
 - `max_depth` $\in \{3, 5\}$,
 - `learning_rate` $\in \{0,05, 0,1\}$,
 - `n_estimators` $\in \{150, 300\}$,
 - `subsample` $\in \{0,8, 1,0\}$,
 - `colsample_bytree` $\in \{0,8, 1,0\}$.
- **Amplia**: extiende a 3 valores por parámetro (e.g. `max_depth` $\in \{3, 5, 7\}$, `learning_rate` $\in \{0,03, 0,05, 0,1\}$, `n_estimators` $\in \{150, 300, 500\}$).
- **GridSearch original**: grilla completa con `max_depth` $\in \{3, 5, 7, 10, 15, 20\}$, `learning_rate` $\in \{0,01, 0,05, 0,1\}$, `n_estimators` $\in \{100, 250, 500, 750, 1000\}$, `subsample` $\in \{0,5, 0,8, 1,0\}$, `colsample_bytree` $\in \{0,5, 0,8, 1,0\}$.

Los parámetros de regularización (`min_child_weight`, `gamma`, `reg_alpha`, `reg_lambda`) solo se agregan a la búsqueda cuando el flag `use_regularization` está habilitado.

El backend de optimización es configurable: `gridsearch` (por defecto, usa `GridSearchCV`) u `optuna` (búsqueda bayesiana con presupuesto de trials configurable, por defecto 24).

Una vez elegido k^* , `_paper_fit_final_xgb_model` reentrena sobre todo el split de entrenamiento, re-optimiza hiperparámetros solo en train usando el mismo backend y perfil, y evalúa una única vez sobre el test fijo.

4.3.7. Etapa 7: Consolidación por ruta

Cuando M1–M3 terminan, `_paper_run_route` consolida:

- una tabla comparativa por modelo,
- la tabla de métricas por clase usada por el paper,
- las predicciones unificadas sobre los mismos IDs de test,
- la tabla de búsqueda de k utilizada para las visualizaciones de M3.

Estos payloads de ruta se guardan tanto como archivos tabulares como en formato pickle para recarga desde checkpoints.

4.3.8. Etapa 7b: Ruta de actualización de embeddings (opcional)

Cuando está habilitada, la ruta `update_embeddings` (`_paper_build_update_embeddings_dataset`) toma el dataset congelado y reemplaza solo sus columnas de embeddings con embeddings recién extraídos del Transformer fine-tuneado actual, manteniendo las variables de flujo originales intactas. Esta ruta es útil para medir el impacto de un nuevo checkpoint de Transformer sin reconstruir todo el pipeline raw. Sigue las mismas etapas de ejecución de modelos que las rutas frozen y raw.

4.3.9. Etapa 8: Comparación entre frozen y raw

`_paper_compare_routes` compara ambas rutas bajo una tolerancia estricta:

- las cantidades discretas deben coincidir exactamente (por ejemplo, k seleccionado, conteos de filas, conteos de clases y falsos negativos),
- las métricas numéricas pueden diferir como máximo en 0,001.

Si la ruta raw queda bloqueada, o si la comparación falla, toda la réplica queda marcada como `blocked` para efectos de exportación.

4.3.10. Etapa 9: Staging y promoción a LaTeX

El código siempre genera assets candidatos para el paper a partir de la ruta `frozen`:

- `accuracy_vs_k.png`,
- `f1_score_vs_k.png`,
- `false_negatives_pct_vs_k.png`,
- `validation_score_vs_k.png`,
- `metrics.png`,
- `gridsearch_k.tex`,
- `metricas_modelos.tex`.

La promoción hacia `NLP/Latex/images/` y `NLP/Latex/generated/` ocurre solo cuando la comparación frozen/raw aprueba.

4.3.11. Etapa 10: Sincronización al registry

Después de una corrida exitosa o de una corrida completada pero bloqueada, `_persist_paper_replication_pay` escribe artefactos resumen en los stages del registry:

- `paper_replication_frozen`,
- `paper_replication_raw`,
- `paper_replication_compare`,
- `paper_replication_export`.

Esa sincronización es idempotente y se omite cuando `manifest.json` ya indica `registry_sync.completed = true`.

4.4. Contraste con el manuscrito

- **Reutilización del encoder vs. narrativa de entrenamiento del paper.** El manuscrito describe el fine-tuning de BERT en español y luego el uso de la representación [CLS]. La réplica en código no vuelve a entrenar BERT durante la reproducción; en cambio, reutiliza el último clasificador fine-tuneado compatible del registry y extrae embeddings desde su encoder.
- **El preprocesamiento textual no es idéntico.** El manuscrito declara lowercasing más limpieza de espacios antes de tokenizar. En código, la ruta Transformer limpia espacios, pero no fuerza lowercasing en `generate_text_embeddings`; el lowercasing solo está fijado explícitamente en la ruta alternativa TF-IDF.
- **La reducción del embedding se operacionaliza como top-200.** El manuscrito presenta una reducción genérica desde $d = 768$ a p . La implementación fija el bloque narrativo reducido en 200 dimensiones para igualar el dataset congelado y el total publicado de 632 variables.
- **La lógica de ranking es específica por modalidad.** El manuscrito presenta un procedimiento de selección de variables centrado en el espacio multimodal. El código aplica ranking con Random Forest dentro de cada experimento modal: un ranking para M1, otro para M2 y otro para M3. El ranking se realiza sobre datos pre-SMOTE para evitar filtración por muestras sintéticas.
- **El paper reporta la grilla de M3, pero el código busca k para los tres modelos.** Solo la búsqueda de M3 se exporta a figuras y tablas del paper, pero la implementación ejecuta el loop de k para M1, M2 y M3.
- **La tasa operacional de falsos negativos difiere de la definición convencional.** En `_classification_metrics`, `false_negative_rate_positive_class` se calcula como FN/N , donde N es el tamaño total del conjunto evaluado, no como FN/P , donde P es el número de positivos. Esto importa porque el manuscrito habla de “FNRate” sin exponer ese detalle de implementación.
- **SMOTE se aplica dentro de la búsqueda interna de hiperparámetros, no como paso previo separado.** El manuscrito no destaca SMOTE. En código, el balanceo se optimiza conjuntamente con los hiperparámetros de XGBoost dentro de `_paper_optimize_xgb_model`, ajustando tanto parámetros de SMOTE (`sampling_strategy`, `k_neighbors`) como parámetros del modelo.
- **El split holdout es temporal, no puramente estratificado.** El manuscrito implica un split aleatorio estratificado. El código usa `split_mode = "Temporal"`: las filas se ordenan por `accidente_time` y el último 20% se convierte en test. El split estratificado se usa solo como fallback cuando la columna temporal no está disponible o produce splits con una sola clase.
- **La regla de selección de k^* cambió de beneficio marginal a mejor puntaje.** El manuscrito describe una regla de parada por beneficio marginal con tolerancia ε . La implementación actual selecciona el k con el mayor `validation_score`; los empates se resuelven por el k más pequeño.
- **Múltiples perfiles de tuning y backends de optimización disponibles.** El manuscrito describe una única grilla de hiperparámetros. El código soporta tres perfiles (`Rapida`, `Amplia`, `GridSearch original`) con espacios de búsqueda progresivamente más grandes, y dos backends (`GridSearchCV`, `Optuna`).

- **Los parámetros de regularización son opcionales.** El manuscrito no distingue regularización de hiperparámetros base. En código, `min_child_weight`, `gamma`, `reg_alpha` y `reg_lambda` solo se buscan cuando el flag `use_regularization` está habilitado.
- **La validación en tres rutas y los checkpoints son salvaguardas solo del software.** El manuscrito presenta un único flujo analítico. El código agrega una ruta base congelada, una ruta de reconstrucción raw, una ruta opcional de actualización de embeddings, chequeos de paridad entre rutas y checkpoints reanudables para evitar publicar assets del paper sin validación cruzada interna.

4.5. Estructura de checkpoints y artefactos

La estructura durable bajo `Resultados/nlp_in_severity/paper_replication/<run_id>/` es:

- `manifest.json`: estado del run, snapshot del protocolo, fingerprints de insumos, índice de pasos y flag de sincronización al registry.
- `live_status.json`: último estado visible de ejecución.
- `live_events.jsonl`: log append-only de eventos.
- `frozen/models/M1, M2, M3`: resúmenes finales, rankings, resultados de la búsqueda de k , predicciones.
- `raw_build/`: features reconstruidos, métricas granulares, embeddings, tablas de ranking, columnas narrativas seleccionadas y dataset final.
- `raw/models/M1, M2, M3`: misma estructura que la ruta congelada.
- `update_emb_build/`: embeddings re-extraídos sobre las variables de flujo congeladas.
- `update_emb/models/M1, M2, M3`: misma estructura que la ruta congelada (solo presente cuando la ruta de actualización de embeddings está habilitada).
- `compare/`: comparación estricta frozen/raw.
- `export/`: assets candidatos y promovidos para LaTeX, además del payload final serializado.

4.6. Cómo leer una corrida

Al revisar una réplica completada, el orden recomendado es:

1. inspeccionar `manifest.json` para determinar si el run quedó `completed`, `blocked` o `failed`;
2. inspeccionar `frozen/dataset_validation.json` y `raw/summary.json`;
3. inspeccionar `compare/summary.json` y `compare/diff.csv`;
4. si la comparación pasó, inspeccionar `export/payload.json` y los assets promovidos en `NLP/Latex/images/` y `NLP/Latex/generated/`.

Este orden es más estricto que la narrativa del manuscrito, porque el software trata la concordancia entre rutas como condición previa para actualizar el paper.

5. Drift detection

5.1. Objetivo, alcance y relación con el paper

El módulo `src/drift_detection_app.py` implementa un espacio de trabajo de replicación para el paper *Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining*. El propio archivo declara que concentra la sección completa de *Drift detection* en una sola fuente Python, y organiza la reproducción de la metodología, la construcción de features, la selección de variables, la ejecución experimental y la verificación de cobertura.

El alcance real del módulo va más allá de un detector puntual de drift. Incluye:

- preparación determinista del dataset de crash prediction para el corredor analizado;
- estrategias batch de recalibración anual;
- estrategias adaptativas con ADWIN, ARF y KSWIN;
- persistencia de checkpoints, cachés de tuning y artefactos SMOTE;
- salidas resumidas alineadas con Appendix A.6–A.9 y Figure 7 del paper.

La configuración de referencia del paper se fija con `PAPER_TIME_SPAN = 2018-01-01 to 2024-09-30`. Además, el corredor de foco se restringe al conjunto fijo de pórticos 11, 12, 14, 15 (`DRIFT_FOCUS_PORTICOS`), mientras que el orden de segmentos usado para la ingeniería de variables del artículo es 15→14, 14→12 y 12→11.

5.2. Dependencias y fuentes de datos

El módulo asume como fuente principal de flujos la base `Datos/flujos.duckdb` y la tabla `flujos_duckdb`. Para eventos utiliza archivos CSV bajo `Datos/` procesados con `process_accidentes_df`, y para georreferencia y relación entre pórticos utiliza `Datos/Porticos.csv` a través de `load_porticos`, `find_candidate_porticos` y `get_portico_segments`.

La app también depende de librerías opcionales según la etapa:

- `duckdb` para consulta y persistencia de features;
- `optuna` para tuning de hiperparámetros;
- `imblearn` para SMOTE;
- `river` para ADWIN, KSWIN y `AdaptiveRandomForestClassifier`.

5.3. Mapa de la interfaz tab a tab

La función `main()` crea cinco tabs con etiquetas reales `Eventos`, `Feature engineering`, `Feature Selection`, `Experiments` y `Neural drift`. Cada una corresponde a un bloque funcional explícito:

Eventos `_render_events_tab()` carga uno o múltiples archivos de accidentes desde `Datos/`, procesa la asignación a pórticos con `process_accidentes_df()`, conserva los eventos excluidos por falta de mapeo y construye un resumen por tramo para comparar con *Crash prediction*.

Feature engineering `_render_feature_engineering_tab()` ofrece tres fuentes: cargar DuckDB existente, recalcular features o reutilizar el dataset en memoria. En modo de cálculo, consulta `flujos.duckdb`, genera variables para pórticos, segmentos y carriles, construye `target`, aplica Stage 1 y Stage 2 y exporta un DuckDB con tablas `raw_features` y `clean_features`.

Feature Selection `_render_feature_selection_tab()` ejecuta filtrado por correlación, ranking con Random Forest y persistencia del resultado de selección dentro del mismo archivo DuckDB de features.

Experiments `_render_experiments_tab()` bloquea el conjunto final de variables, permite seleccionar modelos batch, estrategias, variantes ARF/KSWIN, modos de balanceo y semillas secuenciales, y luego invoca `run_recalibration_experiments()` con soporte de *preview*, *resume* y arranque limpio ignorando checkpoints existentes.

Neural drift `_render_neural_drift_tab()` delega en `src/Neural_drift_app.py`. Esta pestaña consume el dataset limpio y el subconjunto de features ya seleccionadas por *Drift detection*, construye ventanas temporales para backtesting corto plazo y compara `fixed`, `recalibration`, `fine_tuning` y `retrain` sobre XGBoost, Torch MLP y Torch MLP + Attention. Además, el runner ya emite resultados para `balance_mode = none` y `balance_mode = smote`; cuando se activa `smote`, los parámetros `sampling_strategy` y `k_neighbors` se buscan sobre train/validation y el oversampling sólo se aplica en entrenamiento, nunca sobre validation ni sobre stream.

5.4. Feature engineering y construcción del target

La ingeniería de variables está concentrada en `_compute_drift_article_features()`, con apoyo de `feature_catalog_table()` y `feature_engineering_reference_table()`. El objetivo es replicar la Table 4 del paper y producir un dataset listo para predicción en grilla de 5 minutos, manteniendo exactamente la semántica operativa del código.

Notación operativa Sea $\Delta = 5$ minutos el ancho del intervalo temporal, y sea

$$b(\tau) = \left\lfloor \frac{\tau}{\Delta} \right\rfloor \Delta$$

la función que asigna un timestamp τ al inicio del intervalo correspondiente. Bajo la configuración artículo por defecto, el corredor usa el orden de pórticos

$$P = (15, 14, 12, 11),$$

las categorías

$$C = \{\text{Light}, \text{Heavy}, \text{Motorcycle}\},$$

y los segmentos consecutivos

$$S = \{(15, 14), (14, 12), (12, 11)\}.$$

Cada registro AVI crudo aporta timestamp τ_r , velocidad puntual v_r , categoría c_r , patente π_r , pórtico p_r y carril ℓ_r . La app mapea CATEGORIA a Light/Heavy/Motorcycle con DRIFT_ARTICLE_CATEGORY_MAP, es decir, $\{1 \mapsto \text{Light}, 2 \mapsto \text{Heavy}, 3 \mapsto \text{Heavy}, 4 \mapsto \text{Motorcycle}\}$.

Espacio de variables generado Con el corredor fijo de cuatro pórticos, tres segmentos, tres categorías y tres carriles, la configuración completa genera 260 predictores numéricos antes de agregar **target**:

- 96 variables por pórtico y tipo de vehículo: 4 métricas base + 4 deltas, para 4 pórticos y 3 categorías;
- 8 variables composicionales por pórtico: **ft_motorcycle_*** y **ft_heavy_***;
- 72 variables por segmento y categoría: 5 métricas base + 3 deltas, para 3 segmentos y 3 categorías;
- 21 variables globales por segmento: 4 métricas base + 3 deltas, para 3 segmentos;
- 63 variables por carril de origen: 4 métricas base + 3 deltas, para 3 segmentos y 3 carriles.

Si el usuario restringe pórticos o categorías desde la UI, este espacio se reduce, pero las definiciones matemáticas se mantienen invariantes.

Variables por pórtico y tipo de vehículo Para cada intervalo t , pórtico $p \in P$ y categoría $c \in C$, se define el conjunto de observaciones

$$G_{t,p,c} = \{r : b(\tau_r) = t, p_r = p, c_r = c\}.$$

Sobre ese conjunto se calculan las variables con prefijos **flow_**, **vel_**, **sd_**, **den_**:

$$\begin{aligned} \text{Flow}_{t,p,c} &= |G_{t,p,c}|, \\ \text{Vel}_{t,p,c} &= \frac{1}{|G_{t,p,c}|} \sum_{r \in G_{t,p,c}} v_r, \\ \text{Sd}_{t,p,c} &= \sqrt{\frac{1}{|G_{t,p,c}| - 1} \sum_{r \in G_{t,p,c}} (v_r - \text{Vel}_{t,p,c})^2}, \\ \text{Den}_{t,p,c} &= \begin{cases} \frac{\text{Flow}_{t,p,c}}{\text{Vel}_{t,p,c}}, & \text{si } \text{Vel}_{t,p,c} > 0, \\ \text{NA}, & \text{en otro caso.} \end{cases} \end{aligned}$$

En la implementación real, Flow se rellena con 0 cuando no hay observaciones, mientras que Vel, Sd y Den permanecen vacías cuando el grupo no existe o no tiene suficiente soporte. Además, Sd usa la desviación estándar muestral de **pandas**; por tanto, si sólo hay un vehículo en el grupo, la columna queda en **NaN**.

Los cambios temporales se construyen con primera diferencia dentro de cada serie temporal:

$$\Delta X_{t,p,c} = X_{t,p,c} - X_{t-\Delta,p,c}, \quad X \in \{\text{Flow}, \text{Vel}, \text{Den}, \text{Sd}\}.$$

Esto produce las columnas **delta_flow_***, **delta_vel_***, **delta_den_*** y **delta_sd_***. Para el primer intervalo donde la variable base está definida, el código fija el delta en 0; si la variable base ya nace vacía, el delta también queda vacío.

Fraciones composicionales por p rtico Sobre el mismo intervalo y p rtico, la app calcula el flujo total

$$\text{Flow}_{t,p}^{\text{tot}} = \text{Flow}_{t,p,\text{Light}} + \text{Flow}_{t,p,\text{Heavy}} + \text{Flow}_{t,p,\text{Motorcycle}}.$$

Con ello deriva las fracciones

$$\text{FtMotorcycle}_{t,p} = \frac{\text{Flow}_{t,p,\text{Motorcycle}}}{\text{Flow}_{t,p}^{\text{tot}}}, \quad \text{FtHeavy}_{t,p} = \frac{\text{Flow}_{t,p,\text{Heavy}}}{\text{Flow}_{t,p}^{\text{tot}}}.$$

Estas son las columnas `ft_motorcycle_*` y `ft_heavy_*`. Si el flujo total del p rtico es cero, ambas fracciones quedan en NaN en vez de forzarse a cero, porque el denominador no representa una mezcla observada.

Matching por segmento entre p rticos consecutivos Las variables segmentales no se calculan a partir de velocidades locales del p rtico, sino de viajes emparejados entre dos p rticos consecutivos. Para cada segmento $s = (p^-, p^+) \in S$, la distancia d_s se obtiene con `_lookup_article_segment_distance_km()`, priorizando metadatos viales y usando fallback geom trico si hace falta.

La ventana m xima de matching depende de la distancia:

$$\tau_s = \max\left(30, \frac{60 d_s}{20}\right) \text{ minutos},$$

porque `DRIFT_ARTICLE_MIN_MATCH_WINDOW_MINUTES` = 30 y la velocidad de referencia m nima es 20 km/h. Un par upstream-downstream se acepta si:

- comparte la misma patente π ;
- el paso downstream ocurre despu s del upstream y antes de τ_s ;
- la categor a coincide entre ambos extremos;
- el tiempo de viaje es estrictamente positivo.

La implementaci n usa `merge_asof(..., direction="backward")` desde el p rtico downstream hacia el upstream y luego elimina duplicados por `plate_clean` y `time_start`, conservando el primer downstream compatible de cada observaci n upstream. El intervalo del match se ancla al timestamp upstream, es decir, a $b(\tau_{\text{start}})$.

Para cada match m del segmento s , se define el tiempo de viaje

$$\text{TT}_m = \tau_m^{\text{end}} - \tau_m^{\text{start}},$$

y la velocidad espacial

$$\text{SegVel}_m = \begin{cases} \frac{d_s}{\text{TT}_m/1 \text{ hora}}, & \text{si } d_s > 0, \\ \text{NA}, & \text{si no hay distancia v lida.} \end{cases}$$

En paralelo, se calcula el indicador de cambio de carril

$$\text{LC}_m = \mathbf{1}\{\ell_m^{\text{start}} \neq \ell_m^{\text{end}}\},$$

si ambos carriles existen; en caso contrario, el match aporta NaN a esa componente.

Variables por segmento y tipo de vehículo Sea $M_{t,s,c}$ el conjunto de matches del intervalo t , segmento s y categoría c . A partir de ellos se generan columnas como `flow_light_15_14`, `vel_heavy_14_12` o `cl_motorcycle_12_11`:

$$\begin{aligned} \text{Flow}_{t,s,c} &= |M_{t,s,c}|, \\ \text{Vel}_{t,s,c} &= \frac{1}{|M_{t,s,c}|} \sum_{m \in M_{t,s,c}} \text{SegVel}_m, \\ \text{Sd}_{t,s,c} &= \sqrt{\frac{1}{|M_{t,s,c}| - 1} \sum_{m \in M_{t,s,c}} (\text{SegVel}_m - \text{Vel}_{t,s,c})^2}, \\ \text{CL}_{t,s,c} &= \frac{1}{|M_{t,s,c}^{LC}|} \sum_{m \in M_{t,s,c}^{LC}} \text{LC}_m, \\ \text{Den}_{t,s,c} &= \begin{cases} \frac{\text{Flow}_{t,s,c}}{\text{Vel}_{t,s,c}}, & \text{si } \text{Vel}_{t,s,c} > 0, \\ \text{NA}, & \text{en otro caso.} \end{cases} \end{aligned}$$

Aquí $M_{t,s,c}^{LC}$ denota el subconjunto con carriles observados en ambos extremos. Los deltas ΔFlow , ΔVel y ΔDen repiten la misma definición de primera diferencia temporal usada a nivel de p rtico. No existe ΔCL en el c digo actual.

Variables globales por segmento y por carril de origen El bloque global del segmento agrega todos los matches del intervalo sin separar por categor a:

$$M_{t,s} = \bigcup_{c \in C} M_{t,s,c}.$$

Sobre $M_{t,s}$ se generan `flow_15_14`, `vel_15_14`, `sd_15_14`, `den_15_14` y sus deltas temporales. Matem ticamente, las f rmulas son id nticas a las del bloque segmental por categor a, pero reemplazando $M_{t,s,c}$ por $M_{t,s}$.

Para capturar heterogeneidad por carril, el mismo matching se reagrupa por carril de origen:

$$M_{t,s,\ell} = \{m \in M_{t,s} : \ell_m^{\text{start}} = \ell\}, \quad \ell \in \{1, 2, 3\}.$$

Sobre ese subconjunto se calculan $\text{Flow}_{t,s,\ell}$, $\text{Vel}_{t,s,\ell}$, $\text{Sd}_{t,s,\ell}$, $\text{Den}_{t,s,\ell}$ y sus deltas, produciendo columnas como `vel_lane2_15_14` o `delta_den_lane3_12_11`. Este bloque es importante porque el propio c digo justifica su inclusi n por la relevancia emp rica de variables por carril en la r plica de Figure 6.

Reglas de borde y consistencia num rica La implementaci n distingue entre ausencia de tr fico y ausencia de medida:

- los flujos se rellenan con 0 cuando el grupo no tiene observaciones;
- velocidades, desviaciones, densidades y *lane change fractions* permanecen en NaN si no existe informaci n suficiente;
- los deltas se calculan sobre la versi n ya pivotada de cada bloque, por lo que respetan el mismo patr n de faltantes;
- cuando el c lculo se hace por lotes en DuckDB, las features est ticas coinciden con el c lculo monol tico, y los deltas se reinician en los bordes de lote con 0 o NaN; esto est  cubierto por `test_drift_batched_flow_features_duckdb_consistency`.

Construcción del target El target se agrega con `_add_interval_accident_target()` después de filtrar accidentes del corredor con `_filter_accidents_for_allowed_porticos()`. Si t_a es el timestamp de un accidente, su marca binaria se asigna al intervalo

$$t^* = b(t_a) - \Delta.$$

Por tanto,

$$y_t = 1 \iff \exists a \text{ tal que } b(t_a) = t + \Delta.$$

Esta definición implementa un *lead time* de un bin: las features de t intentan anticipar accidentes que ocurren en el siguiente intervalo de 5 minutos. La prueba `test_drift_article_features_cover_table4_and_target` valida explícitamente este corrimiento; por ejemplo, un accidente a las 00:06 activa `target = 1` en la fila de las 00:00, no en la de las 00:05.

5.5. Preparación de datos: Stage 1 y Stage 2

La Sección 4.2 del paper se operacionaliza con `apply_missing_data_policy()`, `identify_long_zero_accident_runs()`, `filter_long_zero_accident_runs()` y `run_configurable_preparation_pipeline()`.

- **Stage 1:** elimina variables con más de 1% de valores faltantes y luego elimina intervalos con cualquier predictor faltante en las variables restantes.
- **Stage 2:** elimina ventanas largas sin accidentes, por defecto de al menos 7 días consecutivos con `target = 0`.

La salida incluye un `DataPreparationSummary` con conteo de filas iniciales/finales, variables removidas, ventanas detectadas y filas descartadas. La prueba `test_configurable_preparation_pipeline_respects_filters` valida que ambos stages respondan correctamente a sus selectores.

5.6. Selección de variables

La etapa de selección usa `compute_abs_correlations()`, `drop_highly_correlated_features()` y `rank_features_random_forest()`. Operativamente se ejecuta en dos pasos:

1. remover variables altamente correlacionadas con un umbral configurable;
2. entrenar un Random Forest para ordenar las variables restantes y quedarse con el top- N efectivo para los experimentos.

La misma pestaña genera la réplica de Figure 5 (densidad de $|\rho|$) y Figure 6 (importancias top), y persiste el payload de selección en el DuckDB de features. La prueba `test_feature_selection_payload_persists` cubre esta persistencia.

5.7. Diseño experimental y estrategias

Las estrategias disponibles están definidas por `EXPERIMENT_STRATEGIES`: `static`, `period_aligned`, `cumulative`, `adaptive_adwin`, `adaptive_arf` y `adaptive_kswin`. Los modelos batch soportados por `MODEL_NAMES` son `XGBoost`, `AdaBoost`, `Random Forest` y `NNet`; las variantes online son `ARFmoderate`, `ARFmaj`, `ARFfast`, `KSWINpaper` y `KSWINseasonal`.

Configuración experimental formal Sea $Y_0 < Y_1 < \dots < Y_J$ la secuencia de años observados en el dataset ordenado por `interval_start`. La app toma como año base Y_0 por defecto, salvo que el usuario fije otro. Para cada intervalo t , el par (x_t, y_t) está formado por las features seleccionadas y el target binario de accidente a un paso. El experimento siempre opera sobre el dataset limpio de Stage 1 y Stage 2.

La orquestación principal en `run_recalibration_experiments()` construye bloques experimentales sobre el producto cartesiano de:

- estrategias;
- modelos batch o variantes online;
- modos de balanceo `none/smote` cuando aplica;
- semillas de repetición secuenciales.

Esto significa que el resultado final no es una sola corrida, sino un conjunto de bloques persistidos y reensamblados al final. Formalmente, si B es el conjunto de bloques y R el conjunto de semillas, la cantidad base de ejecuciones es $|B| \times |R|$, a la que se agregan las tareas globales de tuning canónico.

Entrenamiento batch interno y calibración de threshold Las estrategias batch, `adaptive_adwin` y `adaptive_kswin` reutilizan `train_model_with_internal_validation()`. Dado un split de entrenamiento T , el flujo real del código es:

1. dividir T en un split estratificado interno $(T^{\text{fit}}, T^{\text{val}})$ con proporción de validación $\rho = 0,2$ por defecto;
2. sintonizar hiperparámetros sobre T^{fit} con AUC media de validación cruzada estratificada;
3. calibrar el umbral operativo sobre todo T usando scores *out-of-fold*;
4. reentrenar el modelo final con todos los datos de T , y aplicar SMOTE si el bloque lo requiere.

Sea $\Theta_{m,b}$ el espacio de búsqueda del modelo m y del modo de balanceo b . La selección de hiperparámetros resuelve:

$$\hat{\theta} = \arg \max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^K \text{AUC}_k(\theta),$$

donde K es el número efectivo de folds estratificados. Si `balance_mode = smote`, el propio espacio de búsqueda incorpora `sampling_strategy` y `k_neighbors`; si `balance_mode = none`, sólo se buscan hiperparámetros del modelo.

Una vez elegido $\hat{\theta}$, el código genera scores cruzados s_i^{cv} para cada fila $i \in T$ y selecciona el umbral con el criterio de Youden:

$$\hat{\tau} = \arg \max_{\tau} [\text{TPR}(\tau) - \text{FPR}(\tau)] = \arg \max_{\tau} [\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1].$$

Las predicciones duras se definen como

$$\hat{y}_i = \mathbf{1}\{s_i \geq \hat{\tau}\}.$$

Si la calibración *out-of-fold* falla por falta de variabilidad, el código cae a un esquema de respaldo usando el holdout interno T^{val} .

Política de estabilidad para NNet El modelo NNet ya no se entrena sobre variables crudas. Después de la imputación por mediana del split de entrenamiento activo, el código ajusta un `StandardScaler` exclusivamente sobre la partición de entrenamiento correspondiente y transforma validación, test o stream con esos mismos parámetros. Para cada predictor x_j , la transformación aplicada es

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j},$$

donde μ_j y σ_j se estiman sólo con el subconjunto de entrenamiento del fold, del holdout interno o de la ventana de reentrenamiento que esté activa. Esto evita leakage temporal o entre folds, y la misma lógica se reutiliza en evaluación anual, `adaptive_adwin` y `adaptive_kswin`.

En términos de optimización, `MLPClassifier` se ejecuta con `solver = adam`, `early_stopping = True`, `validation_fraction = 0.1`, `n_iter_no_change = 20` y `tol = 1e-4`. Además, la grilla de `maxit` se endurece a `{300,500}` en *fast mode* y a `{300,500,800}` en modo completo. Los `ConvergenceWarning` de `scikit-learn` no se dejan escapar a terminal: se capturan como señal interna del pipeline.

Metodológicamente, un fold de validación cruzada de NNet que no converge se considera inválido y no puede sostener un trial ganador. Si todos los folds relevantes fallan, el artefacto de tuning queda marcado con `has_valid_trial = false`. Durante el refit final, el código intenta una sola vez un reentrenamiento adicional con `max_iter` duplicado, acotado superiormente por 1000. Si ese reintento tampoco converge, el bloque experimental no se aborta: se persiste como `skipped` y el `execution_log` registra eventos explícitos como `nnet_fold_non_converged`, `nnet_trial_invalid`, `nnet_final_refit_retry` o `nnet_final_refit_skipped`. Los trials persistidos también conservan `converged`, `n_non_converged_folds`, `n_valid_folds` y `n_iter_final` para inspección online.

Balanceo con SMOTE El modo `smote` nunca se aplica sobre test o stream, sólo sobre folds de entrenamiento y sobre el refit final. Si la clase minoritaria tiene menos de dos ejemplos, o si la razón minoritaria/mayoritaria ya está por encima del objetivo, el balanceo no se ejecuta. En caso contrario, SMOTE construye un conjunto balanceado $(X^{\text{bal}}, y^{\text{bal}})$ con:

$$\frac{N_{\text{minor}}^{\text{new}}}{N_{\text{major}}} \approx \text{sampling_strategy},$$

y el vecindario efectivo satisface

$$k_{\text{eff}} = \min(k_{\text{neighbors}}, N_{\text{minor}} - 1).$$

Los artefactos SMOTE pueden persistirse y reutilizarse por bloque mediante `_load_or_create_smote_artifact()`.

Estrategias batch anuales `run_yearly_strategy()` implementa tres reglas de entrenamiento para cada año objetivo Y_j , $j \geq 1$:

$$\mathcal{T}_j^{\text{static}} = \{t : \text{year}(t) = Y_0\},$$

$$\mathcal{T}_j^{\text{period}} = \{t : \text{year}(t) = Y_j - 1\},$$

$$\mathcal{T}_j^{\text{cum}} = \{t : \text{year}(t) \leq Y_j - 1\},$$

y el conjunto de evaluación es siempre

$$\mathcal{E}_j = \{t : \text{year}(t) = Y_j\}.$$

Cada fila de salida contiene `training_year`, `prediction_year`, `model`, `balance_mode`, métricas, umbral, tiempo de entrenamiento y tamaños `n_train/n_test`. En la estrategia `static`, como $\mathcal{T}_j^{\text{static}}$ es idéntico para todo j , el código cachea y reutiliza el bundle entrenado para evitar tuning y refit redundantes; esto está cubierto por `test_run_yearly_strategy_reuses_static_training_bundle`.

El resultado esperado de este bloque es:

- una fila en `yearly_results` por combinación (estrategia, modelo, balance, Y_j , seed);
- una curva ROC segmentada por año objetivo en `roc_payload`;
- tablas Appendix A.6, A.7 y A.8 para `static`, `period_aligned` y `cumulative`, respectivamente.

Recalibración adaptativa con ADWIN `run_adaptive_strategy()` entrena primero un modelo batch sobre el año base Y_0 y luego procesa cronológicamente todo el stream posterior. Para cada fila del stream se calcula un score s_t , una predicción dura $\hat{y}_t = \mathbf{1}\{s_t \geq \hat{\tau}\}$ y una señal de error binario

$$e_t = \mathbf{1}\{\hat{y}_t \neq y_t\}.$$

La clase `SimpleADWIN` mantiene una ventana W de errores. Para cada corte candidato $W = W_0 \cup W_1$, con tamaños n_0 y n_1 , define:

$$\mu_0 = \frac{1}{n_0} \sum_{e \in W_0} e, \quad \mu_1 = \frac{1}{n_1} \sum_{e \in W_1} e,$$

$$m = \left(\frac{1}{n_0} + \frac{1}{n_1} \right)^{-1}, \quad \delta' = \frac{\delta}{n}, \quad \epsilon = \sqrt{\frac{1}{2m} \log \left(\frac{4}{\delta'} \right)} = \sqrt{\frac{1}{2m} \log \left(\frac{4n}{\delta} \right)},$$

donde $n = |W|$ y $\delta = \text{adwin_delta}$. Se declara drift si

$$|\mu_0 - \mu_1| > \epsilon.$$

Cuando eso ocurre, el código:

- cierra el segmento de predicción acumulado hasta t ;
- registra `drift_date`, `W`, `W0`, `W1` y `remaining_periods`;
- intenta recalibrar el modelo usando las últimas $|W_1|$ observaciones del stream.

El reentrenamiento sólo ocurre si la ventana W_1 tiene al menos $\max(20, \lfloor \text{min_window}/10 \rfloor)$ filas, o el `min_retrain_size` que fije el usuario, y contiene ambas clases. Si no se cumple esa condición, el detector igual corta el segmento, pero el modelo vigente continúa. El resultado esperado es una tabla `adaptive_results` con una fila por drift detectado más un segmento final con `remaining_periods = 0`; estas filas alimentan la parte ADWIN de Appendix A.9.

Adaptive Random Forest `run_arf_strategy()` implementa una estrategia genuinamente on-line. Sobre el año base Y_0 , la función `train_arf_with_internal_validation()` separa un bloque final de validación cronológica y usa el bloque inicial para *warm-up* del ensamble ARF. Luego predice secuencialmente sobre el bloque de validación, aprende inmediatamente cada etiqueta y fija $\hat{\tau}$ con el mismo criterio de Youden.

Durante el stream posterior, el modelo se actualiza observación a observación:

$$s_t = f_{t-1}(x_t), \quad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

donde $\mathcal{U}$ representa el paso online `learn_one`. A diferencia de ADWIN y KSWIN, no hay un reentrenamiento batch externo tras un drift: la adaptación está internalizada en el bosque. Por eso el código no duplica esta estrategia por `balance_mode`; siempre usa `not_applicable`.

Los resultados se segmentan por año de predicción, no por drift externo. Cada fila esperada contiene:

- `prediction_year`;
- `segment_rows`;
- `n_internal_drifts`;
- `n_internal_warnings`;
- métricas y umbral operativo.

En consecuencia, Appendix A.9 muestra a ARF como segmentos anuales online, no como ventanas $W/W_0/W_1$. Las pruebas `test_run_arf_strategy_variants` y `test_recalibration_experiments_support_adapt` verifican esa salida.

Recalibración adaptativa con KSWIN `run_kswin_strategy()` separa claramente el problema de detección de drift covariable del problema de predicción. Primero entrena un modelo batch sobre Y_0 . Luego selecciona las variables monitorizadas con `_select_kswin_monitor_columns()`, que en la implementación actual toma las primeras **top-k** columnas numéricas del orden de `feature_cols`. No es un ranking online: es una selección determinista de las features ya aprobadas por Feature Selection.

Para cada feature monitorizada f , KSWIN puede operar en dos espacios:

- **KSWINpaper**: usa el valor crudo $x_{t,f}$;
- **KSWINseasonal**: usa un z-score estacional

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

con fallback a perfiles por hora y luego a perfiles globales del año base.

Cada detector usa $\alpha = 10^{-4}$, `window_size` = 300 y `stat_size` = 30, por lo que Appendix A.9 reporta

$$W = 300, \quad W_0 = 270, \quad W_1 = 30$$

como tamaños fijos del detector, no como ventanas aprendidas dinámicamente.

Sea F_t el conjunto de features cuyo detector dispara drift en el instante t . La condición de drift del bloque es:

$$|F_t| \geq \text{vote_threshold}.$$

Cuando eso ocurre, se cierra el segmento de predicción, se registran `vote_count`, `detected_features`, `monitored_features`, y se selecciona una ventana de recalibración temporal:

$$\mathcal{R}_t = \{i : t_i \geq t - D\},$$

donde $D = \text{kswin_retrain_days}$. Si $|\mathcal{R}_t| < \text{kswin_min_retrain_rows}$, el código usa las últimas `min_rows` observaciones; si esa ventana tiene una sola clase, se expande a todo lo observado hasta

t. Tras el refit batch, los detectores KSWIN se reconstruyen y se recalientan con la ventana de reentrenamiento.

El resultado esperado de KSWIN es una fila por drift votado más un segmento final, con columnas adicionales como `vote_threshold`, `monitor_feature_count`, `retrain_rows` y `retrain_positive_rows`. Las pruebas `test_run_kswin_strategy_variants`, `test_recalibration_experiments_support_adaptive_kswin` y `test_kswin_optuna_studies_capture_base_and_retrains_with_balance_mode` validan precisamente ese comportamiento.

Semillas, tuning global, persistencia y reanudación `run_recalibration_experiments()` no vuelve a tunear desde cero cada bloque batch. Antes de lanzar los bloques, construye tareas de tuning canónico sobre el split base del año Y_0 para cada combinación (modelo, balance) que vaya a ser usada por estrategias batch, ADWIN o KSWIN. Ese tuning global se persiste en `tuning_dir` y luego se inyecta como `fixed_params/fixed_tuning_artifact` en los bloques posteriores. En términos metodológicos, esto impone comparabilidad entre estrategias que comparten familia de modelo, porque todas parten del mismo $\hat{\theta}$ canónico por seed y modo de balanceo.

Cada bloque persiste:

- filas anuales o adaptativas;
- ROC payload segmentado;
- su propio `execution_log`;
- referencias a tuning y a artefactos SMOTE.

El `manifest.json` central mantiene progreso, bloques completados, bloques omitidos por falla, cachés disponibles y errores. Por defecto, un bloque fallido sigue abortando la corrida y deja un checkpoint reanudable. Opcionalmente, `continue_on_block_error=True` cambia sólo la política a nivel de bloque: el bloque fallido se marca como `skipped_failed`, su error se registra en `nonfatal_block_errors` y la corrida continúa con los bloques restantes. El tuning global sigue siendo *fail-fast*. El soporte de *resume* y de omisión controlada está cubierto por `test_preview_recalibration_checkpoint_detects_resumable_m`, `test_recalibration_experiments_resume_from_failed_checkpoint` y `test_recalibration_experiments_can`.

Para **NNet**, la compatibilidad de caché no depende sólo del dataset y de la grilla. Tanto el `run_id` como el `tuning_key` incorporan una firma de versión de la política de entrenamiento (`NNET_TRAINING_POLICY_VERSION`). Por ello, si cambian el escalado, la estrategia de *early stopping* o las reglas de convergencia, los checkpoints y tuning caches previos dejan de ser compatibles y no se reutilizan de manera silenciosa.

Resultados esperados Metodológicamente, la salida completa del experimento debe contener siete productos coherentes entre sí:

- `yearly_results`: una fila por split anual batch, con `strategy`, `prediction_year`, `model`, `balance_mode`, métricas, umbral y tamaños muestrales;
- `adaptive_results`: una fila por segmento adaptativo, ya sea delimitado por drift ADWIN/KSWIN o por año en ARF;
- `summary`: medias por (`strategy`, `model`, `balance_mode`), incluyendo `n_segments` y `n_repetitions`;
- `appendix_tables`: A.6 para `static`, A.7 para `period_aligned`, A.8 para `cumulative` y A.9 para resultados adaptativos;

- **appendix_tables_mean**: las mismas tablas, pero promediadas por semillas secuenciales y enriquecidas con `seed_list` y `run_orders`;
- **average_roc**: curvas ROC medias tipo Figure 7, construidas interpolando cada curva individual sobre una grilla uniforme de 101 puntos en FPR y promediando la TPR:

$$\overline{\text{TPR}}_g(u) = \frac{1}{J_g} \sum_{j=1}^{J_g} \widetilde{\text{TPR}}_{g,j}(u);$$

- **run_manifest** y **execution_log**: trazabilidad reproducible del experimento, incluyendo configuración, seeds, thresholds, tuning, checkpoints, errores y consumo de recursos.

Las pruebas `test_end_to_end_recalibration_and_average_roc`, `test_recalibration_experiments_compare_b` y `test_recalibration_experiments_persist_optuna_json` verifican precisamente que ese paquete de resultados exista, sea consistente y pueda persistirse como JSON final.

5.8. Neural drift: objetivo, alcance y unidad experimental

El submódulo `src/Neural_drift_app.py` implementa un espacio de backtesting online para predicción de crash a corto horizonte con detección adaptativa de drift. No replica el protocolo anual del paper, sino que lo complementa con una simulación secuencial donde cada fila del stream se procesa en orden temporal, se calcula severidad de drift y, si corresponde, se activa una acción de adaptación sobre el modelo vigente.

La unidad experimental real es

(`model`, `strategy`, `balance_mode`),

por lo que las salidas `baseline`, `summary`, `stream_metrics`, `rolling_metrics`, `drift_events`, `attention_*` y `detector_attention_*` siempre deben leerse por combinación. Esta decisión es importante porque el módulo compara simultáneamente:

- tres familias de predictor: XGBoost, Torch MLP y Torch MLP + Attention;
- cuatro estrategias de respuesta: `fixed`, `recalibration`, `fine_tuning` y `retrain`;
- uno o dos modos de balanceo: `none` y/o `smote`;
- cuatro canales de drift: `input drift`, `score drift`, `error drift` y `embedding drift`.

5.9. Pipeline operacional paso a paso

El flujo efectivo de `run_backtest_pipeline()` es el siguiente:

1. tomar el dataset limpio exportado desde *Drift detection*;
2. reducirlo al tramo más reciente si `dataset_percent < 100`;
3. ampliar la matriz con features derivadas mediante `augment_feature_frame()`;
4. construir ventanas temporales planas con `build_window_dataset()`;
5. partir esas ventanas en `train`, `validation` y `stream`;

6. entrenar un artefacto canónico por `(model, balance_mode)`;
7. clonar ese artefacto y simular cada `strategy` sobre el mismo stream;
8. registrar, por fila, score, predicción, severidad, trigger, adaptación aplicada y metadatos del detector;
9. resumir el stream en tablas agregadas, tablas rolling y resúmenes de attention;
10. opcionalmente persistir toda la corrida con `run_backtest_with_checkpoints()`.

En otras palabras, *Neural drift* separa dos niveles: primero fija una línea base canónica comparable y luego evalúa, sobre exactamente el mismo stream, qué tan bien responde cada estrategia cuando aparecen cambios de régimen.

5.10. Dataset temporal, augmentación y construcción de ventanas

Fuente efectiva El tab consume el dataset limpio de *Drift detection* y exige una columna temporal `interval_start`, una columna objetivo `target` y un conjunto explícito de features seleccionadas. Si la app recibe un DuckDB con `clean_features`, también puede reconstruir el contexto desde el artefacto persistido.

Augmentación de features La función `augment_feature_frame()` parte del subconjunto $\mathcal{F}$ de features base y, para cada variable $x_t \in \mathcal{F}$, agrega cinco derivados:

$$\Delta x_t = x_t - x_{t-1}, \quad \text{rate}(x_t) = \frac{x_t - x_{t-1}}{|x_{t-1}|},$$

$$\bar{x}_t^{(3)}, \quad s_t^{(3)}, \quad \text{cv}_t^{(3)} = \frac{s_t^{(3)}}{|\bar{x}_t^{(3)}|},$$

donde las estadísticas rolling usan ventana 3 y `min_periods = 2`. A eso se suman variables agregadas de régimen:

- `shock_speed_drop`: caída positiva de la velocidad media;
- `shock_density_jump`: salto positivo de la densidad media;
- `speed_level`, `density_level`;
- `flow_density_ratio`, `speed_density_interaction`, `relative_congestion`;
- `heavy_speed_interaction`;
- `hour_sin`, `hour_cos`, `day_of_week`, `is_weekend`.

Esta capa no sustituye las variables de *Drift detection*; las encapsula dentro de un espacio más sensible a cambios locales, pendiente temporal y contexto horario.

Etiqueta futura y ventanas La etiqueta efectiva se construye con `_future_target_from_interval_target()`. Si y_t es el **target** del dataset de entrada y $H = \text{horizon_steps}$, la ventana que termina en t recibe

$$\tilde{y}_t = \text{máx}\{y_t, y_{t+1}, \dots, y_{t+H-1}\}.$$

Con la configuración por defecto $H = 1$, $\tilde{y}_t = y_t$. Como el **target** upstream ya representa un accidente adelantado un bin de 5 minutos, la semántica operativa por defecto sigue siendo “predecir el siguiente intervalo inmediato”.

Luego, `build_window_dataset()` recorre el dataframe ordenado y forma una ventana plana de longitud

$$L = \text{lookback_steps},$$

apilando las L observaciones más recientes para producir un vector de dimensión

$$d_{\text{input}} = L \times |\mathcal{F}_{\text{aug}}|.$$

Cada muestra conserva metadatos: `prediction_time`, `window_start`, `window_end`, `horizon_end` y el `sample_index`.

Splits temporales El módulo soporta dos esquemas:

- **fractions**: divide secuencialmente por proporciones `train_fraction`, `validation_fraction` y deja el resto como `stream`;
- **fixed_dates**: usa una base histórica fija y un stream posterior, con `base_start`, `base_end` y `stream_start`.

En el modo fraccional, la app exige al menos 10 muestras para entrenamiento, 5 para validación y 5 para stream. En **fixed_dates**, la configuración usada por los experimentos fija:

$$\text{base_start} = 2018-01-01, \quad \text{base_end} = 2018-12-31, \quad \text{stream_start} = 2019-01-01.$$

Sobre la base 2018, la cola temporal se reserva como validation y el resto queda como train. Este diseño fuerza un protocolo *train on past, score on future* sin mezcla aleatoria entre periodos.

5.11. Configuración y parámetros principales

La configuración base se concentra en `DEFAULT_CONFIG`. Los grupos operativos más relevantes son los siguientes.

Parámetro	Default	Rol operacional
<code>interval_minutes</code>	5	Ancho del bin temporal.
<code>dataset_percent</code>	100	Usa el tramo más reciente del dataset si se reduce el porcentaje.
<code>lookback_steps</code>	12	Largo de la ventana histórica de entrada.
<code>horizon_steps</code>	1	Horizonte binario agregado sobre el target .
<code>train_fraction</code>	0.60	Proporción temporal para entrenamiento en modo fraccional.
<code>validation_fraction</code>	0.20	Proporción temporal para validación.
<code>max_stream_rows</code>	600	Recorta el stream a las filas más recientes si es necesario.
<code>rolling_metric_window</code>	48	Ventana de suavizado para métricas rolling.
<code>random_state</code>	42	Semilla global del backtest.

Parámetro	Default	Rol operacional
balance_modes	[none]	Ejecuta none y/o smote .
models	3 modelos	XGBoost, Torch MLP, Torch MLP + Attention.
strategies	4 estrategias	fixed , recalibration , fine_tuning , retrain .
recent_window_size	96	Ventana reciente usada por el detector y por la adaptación local.
recalibration_min_rows	32	Mínimo para recalibrar.
retrain_min_rows	64	Mínimo para reentrenar.
history_sample_size	256	Historia conservada al construir el bloque híbrido de retrain .
threshold_beta	2.0	β usado al optimizar el umbral operativo.

Parámetro	Default	Rol operacional
xgb_estimators	80	Booster base de XGBoost.
xgb_parallel_neural_enabled	True	Activa una rama neuronal auxiliar paralela para XGBoost.
xgb_fine_tune_selection_metric	β	Métrica de selección del mejor <i>fine-tuning</i> de XGBoost.
xgb_fine_tune_selection_beta_recall	0.75	
xgb_fine_tune_window_min/max	32 / 160	Rango de filas recientes elegibles para el <i>fine-tuning</i> .
xgb_fine_tune_rounds_min/max	2 / 24	Rango de rondas incrementales.
xgb_fine_tune_eta_multiplier_max	1.5	Multiplicador máximo del learning rate base según severidad.
xgb_fine_tune_recent_weight_max	1.0	Peso máximo para las filas más recientes.
mlp_hidden_dim	96	Capa oculta principal del predictor Torch.
mlp_embedding_dim	24	Tamaño del embedding del predictor.
mlp_dropout	0.10	Regularización del predictor Torch base.
mlp_epochs / mlp_batch_size	20 / 64	Presupuesto de entrenamiento Torch.
mlp_learning_rate	10^{-3}	Learning rate base del predictor Torch.
fine_tune_learning_rate	/ 3×10^{-4} / 6	Ajuste fino de modelos Torch.
fine_tune_epochs		

Parámetro	Default	Rol operacional
drift_channels	4 canales	Canales activos de drift.
severity_threshold	0.50	Umbral final del evento de drift.
detector_adwin_delta	0.002	Sensibilidad de los detectores clásicos basados en ADWIN.
drift_point_signal_weight	1.0	Peso relativo entre picos puntuales y promedio de ventana.
drift_monitor_architecture	Attention temporal	Arquitectura solicitada para el detector neuronal.
drift_monitor_hidden_dim	16	Capa oculta del monitor.
drift_monitor_bottleneck_dim	6	Severidad del cuello de botella del monitor.
drift_monitor_reconstruction_weight	0.05	Peso del error de reconstrucción dentro de embedding drift .
drift_monitor_sequence_length	12	Largo de secuencia del monitor temporal.
drift_monitor_attention_hidden_dim	32	Capacidad de la attention del monitor temporal.
attention_feature_hidden_dim	32 / 32	Dimensiones ocultas del predictor con attention.
/ attention_temporal_hidden_dim		
attention_dropout	/ 0.10 / 8	Regularización del predictor con attention y tamaño del resumen visual.
attention_top_k		

Perfiles del monitor y presets de sensibilidad El monitor neuronal expone dos perfiles:

- Moderado: `drift_monitor_bottleneck_dim = 6` y `drift_monitor_reconstruction_weight = 0.65`;
- Sensible: `drift_monitor_bottleneck_dim = 4` y `drift_monitor_reconstruction_weight = 0.85`.

La UI expone además cuatro presets de sensibilidad del detector:

Preset	τ	recent_window	ADWIN delta	Efecto
Conservador	0.65	120	0.001	Menos falsas alarmas y menor peso puntual.
Moderado	0.50	96	0.002	Configuración base.
Sensible	0.40	48	0.004	Mayor reactividad y perfil sensible.
Muy sensible	0.30	24	0.010	Máxima reactividad, con mayor riesgo de disparos tempranos.

5.12. Modelos predictivos, detector neuronal y drift channels

Modelos predictivos WindowMLP procesa la ventana plana con un encoder denso

$$d_{\text{input}} \rightarrow \text{hidden_dim} \rightarrow \frac{\text{hidden_dim}}{2} \rightarrow \text{embedding_dim} \rightarrow 1,$$

mientras WindowAttentionMLP reordena la entrada como

$$[\text{batch}, \text{lookback_steps}, \text{feature_count}],$$

y aplica una attention sobre features dentro de cada paso temporal y otra sobre los pasos del lookback antes de producir el embedding final. Ambos modelos usan BCEWithLogitsLoss, Adam, StandardScaler y parada temprana sobre validation.

Para XGBoost, `_train_xgboost_model()` imputa con medianas, entrena un `XGBClassifier` con `tree_method = hist` y calcula calibración de Platt + umbral óptimo sobre validation. Si `xgb_parallel_neural_enabled = True`, el booster mantiene además una Torch MLP auxiliar cuya salida no reemplaza al predictor oficial, pero sí alimenta al detector.

Referencia estadística Cada artefacto guarda estadísticas de referencia estimadas sobre validation:

- medias y desviaciones por feature;
- media y dispersión del score calibrado;
- media y dispersión del error cuadrático $(\hat{p} - y)^2$;
- si existen embeddings, centroide latente, distancias al centroide y estadísticos de reconstrucción.

Monitor neuronal El detector neuronal tiene dos variantes:

- **autoencoder clásico:** $\mathbf{e}_t \rightarrow \text{hidden} \rightarrow \text{bottleneck} \rightarrow \hat{\mathbf{e}}_t$;
- **attention temporal:** toma una secuencia reciente de embeddings, aprende pesos temporales, forma un contexto latente y reconstruye el embedding objetivo actual.

Si la variante temporal no tiene secuencias suficientes, la app hace *fallback* al autoencoder clásico. Si tampoco hay soporte para eso, el monitor queda ausente.

Canales de drift La normalización compartida de las señales es

$$\text{norm}(v; \mu, \sigma) = \text{clip} \left(\frac{|v - \mu|}{3 \max(\sigma, 10^{-6})}, 0, 1 \right).$$

Sobre esa base, el submódulo define:

- **input drift:** z-score absoluto medio de la fila imputada respecto a las covariables de referencia;
- **score drift:** desviación del score calibrado del predictor principal y, si existe, de la rama neuronal paralela;
- **error drift:** desviación del error cuadrático instantáneo;
- **embedding drift:** combinación de distancia al centroide latente y error de reconstrucción del monitor.

Si $\lambda = \text{drift_point_signal_weight}$, el score final de cada canal mezcla lectura puntual y promedio reciente:

$$s_{\text{canal}} = (1 - \lambda)s_{\text{window}} + \lambda \max(s_{\text{point}}, s_{\text{window}}).$$

En los canales clásicos, si ADWIN dispara drift, el score se satura al menos en 1,0. En **embedding drift** no se usa ADWIN; el flag se activa por umbral directo sobre el score y sus componentes.

Severidad global Si C_t es el conjunto de canales activos en el instante t , la app calcula

$$\text{severity_score}_t = \frac{1}{|C_t|} \sum_{c \in C_t} c, \quad \text{max_channel_score}_t = \max_{c \in C_t} c.$$

Se registra un evento si

$$\text{severity_score}_t \geq \tau \vee \text{max_channel_score}_t \geq \tau,$$

donde $\tau = \text{severity_threshold}$. La intensidad continua usada por XGBoost *fine-tuning* es

$$\text{severity_intensity}_t = \text{clip} \left(\frac{\max(\text{severity_score}_t, \text{max_channel_score}_t) - \tau}{1 - \tau}, 0, 1 \right).$$

5.13. Strategies y optimizaciones de adaptación

fixed El predictor nunca se modifica. El sistema sigue midiendo severidad y registrando **drift_events**, pero **action_taken** permanece en **none**. Esta estrategia es el control negativo del backtest.

recalibration Si hay trigger, la ventana reciente tiene al menos `recalibration_min_rows` filas y contiene ambas clases, la app:

1. recalcula scores sobre la ventana reciente;
2. reentrena el calibrador de Platt;
3. reoptimiza el umbral operativo con F_β ;
4. reconstruye las estadísticas de referencia y, si aplica, el monitor neuronal.

No se cambian los pesos del predictor. En modelos **XGBoost** con rama paralela, la recalibración también se propaga a la rama auxiliar.

fine_tuning Para modelos Torch, el ajuste fino reutiliza `base_model_state`, baja el learning rate a `fine_tune_learning_rate`, reduce el presupuesto a `fine_tune_epochs` y reentrena sobre la ventana reciente. Para **XGBoost**, la lógica es más rica: la app toma todo el stream observado hasta el instante actual, selecciona una subventana adaptativa dentro de `xgb_fine_tune_window_min/max`, la expande de a 8 filas hasta obtener train y validation con ambas clases, y luego optimiza:

- el learning rate incremental:

$$\eta_t = \eta_{\text{base}} [1 + (\text{xgb_fine_tune_eta_multiplier_max} - 1) \text{severity_intensity}_t];$$

- los pesos de recencia, crecientes entre 1 y

$$1 + (\text{xgb_fine_tune_recent_weight_max} - 1) \text{severity_intensity}_t;$$

- la cantidad de rondas nuevas, elegida dentro de `xgb_fine_tune_rounds_min/max`.

Cada candidata se evalúa con una métrica seleccionable (`f_beta_recall`, `pr_auc`, `brier`, `f1`, `roc_auc`, `balanced_f1` o `mcc`) y la mejor se persiste en columnas como `xgb_adaptation_window_rows`, `xgb_fine_tune_rounds` y `xgb_fine_tune_selection_score`.

retrain Si la ventana reciente supera `retrain_min_rows` y contiene ambas clases, la estrategia construye un dataset híbrido:

$$\mathcal{H}_t = \text{últimas } \min(|\text{history}|, \max(|W_t|, \text{history_sample_size})) \text{ filas de historia } \cup W_t.$$

Con ese bloque reentrena un artefacto completo desde cero y luego refresca calibrador, referencia y monitor. La historia fija proviene del `train + validation` original; la ventana reciente proviene del stream hasta el instante actual.

Condiciones de seguridad En todas las estrategias adaptativas, si hay trigger pero la ventana local no tiene ambas clases o no alcanza el tamaño mínimo, el evento igual se registra y `action_taken` queda en `none`. Esto evita adaptaciones inestables con señal insuficiente.

5.14. Backtest: diseño, métricas y salidas

Línea base canónica Antes de simular estrategias, la app entrena un `canonical_artifact` por cada par (`model`, `balance_mode`). Ese artefacto se evalúa una sola vez sobre validation y genera la tabla `baseline`, con métricas de clasificación, umbral operativo, disponibilidad del canal embedding, arquitectura efectiva del monitor y metadatos de SMOTE o de la rama neuronal paralela.

stream_metrics La tabla principal del backtest tiene una fila por timestamp de stream. Registra:

- `timestamp`, `model`, `strategy`, `balance_mode`;
- `y_true`, `prediction`, `score`, `decision_threshold`;
- `severity_score`, `severity_intensity`, `max_channel_score`, `severity_label`;
- `action_taken`;
- `brier_component`, metadatos del monitor y, cuando corresponde, metadatos del *fine-tuning* de XGBoost.

drift_events Cada vez que el trigger se activa, la app agrega una fila a `drift_events`. Allí quedan serializados `channel_scores`, `raw_channel_values`, `detector_flags`, `recent_rows`, `recent_positive_rows` y `action_taken`. Esta es la tabla correcta para auditar de dónde vino un trigger y por qué la respuesta final fue `none`, `recalibration`, `fine_tuning` o `retrain`.

rolling_metrics `_rolling_metric_table()` resume el stream en una ventana deslizante de largo `rolling_metric_window`. Para cada timestamp calcula `pr_auc`, `recall`, `fnr`, `brier`, `severity_score` y `decision_threshold`. Esta tabla mide evolución local, no sólo promedio global.

summary `_summary_from_stream()` colapsa cada grupo (`model`, `strategy`, `balance_mode`) en una fila con `roc_auc`, `pr_auc`, `f1`, `recall`, `specificity`, `fnr`, `brier`, `n_drift_events`, `n_actions`, `mean_severity`, `max_trigger_score`, umbrales medios/finales y metadatos del monitor. `summary` responde “cómo terminó la estrategia”; `rolling_metrics` responde “cómo fue evolucionando”.

Tablas de attention Si el predictor usa Torch MLP + Attention, la app agrega:

- `attention_feature_summary`: peso medio por feature;
- `attention_temporal_summary`: peso medio por paso temporal;
- `attention_drift_shift_summary`: diferencia entre attention de referencia y attention en filas con drift.

Si el monitor neuronal usa `Attention temporal`, agrega también `detector_attention_temporal_summary` y `detector_attention_drift_shift_summary`. Estas tablas no miden exactitud, sino qué partes de la ventana y del historial latente pasaron a explicar el estado actual cuando hubo drift.

5.15. Checkpointing, reanudación y ejecución en vivo

`run_backtest_with_checkpoints()` convierte el backtest en una corrida persistente bajo `Resultados/neural_d`. Cada corrida crea:

- `manifest.json`: estado global, `run_signature`, `config`, `progreso`, `baseline_index` y `experiment_index`;
- `live_status.json`: heartbeat resumido;
- `live_events.jsonl`: eventos secuenciales;
- `baselines/`: baselines persistidos por `baseline_key`;

- `experiments/`: tablas por `experiment_key`;
- `artifacts/`: resultados ensamblados al final.

La reanudación sólo es válida si el `run_signature` coincide exactamente con la configuración y con la fuente de datos activas. Si una corrida ya está completa, la app recarga los artefactos ensamblados; si quedó a medias, continúa sólo con baselines o experimentos pendientes.

5.16. Experimentos: protocolo fijo, fases, score y criterio de ganador

El segundo nivel del sistema vive en `src/neural_drift_experiments.py`. No reemplaza el back-test; lo usa como motor interno para ejecutar un sweep experimental reproducible.

Protocolo temporal fijo El runner exige cobertura completa entre 2018-01-01 y 2024-09-30 y fija una base 2018, un bloque dev 2019–2022 y un holdout 2023–2024. En este protocolo, `dataset_percent` deja de ser una palanca experimental: el runner fuerza 100 % del dataset elegible y usa `split_mode = fixed_dates`.

Seeds, baseline y estudios El sweep usa tres semillas, `EXPERIMENT_SEEDS = (42, 7, 123)`, y siempre corre primero un baseline `cumulative` tomado desde `src/drift_detection_app.py`. Luego compara ese control frente a:

- `study = adwin`: protocolo anual adaptativo clásico;
- `study = neural`: protocolo de *Neural drift*;
- `study = all`: ambos estudios.

Fases y espacios de búsqueda Cada estudio se divide en fases Optuna. Los presupuestos reales son:

- `adwin`: Fase 1 detector 40 trials, Fase 3 optimizador `XGBoost` 20, Fase 4 joint 20;
- `neural`: Fase 1 detector 40, Fase 2 arquitectura 70, Fase 3 optimizador `XGBoost` 30, Fase 4 joint 40.

Las búsquedas más importantes son:

- en `adwin`: `adwin_delta`, `min_window`, `min_retrain_size` y luego hiperparámetros de `XGBoost`;
- en `neural` Fase 1: `detector_adwin_delta`, `recent_window_size`, `severity_threshold`, `lookback_steps`, `rolling_metric_window`;
- en `neural` Fase 2: dimensiones del MLP, embedding, dropout, `attention_top_k`, cuello de botella y arquitectura del monitor, learning rate, epochs y batch size;
- en `neural` Fase 3: métrica de selección y rango del *fine-tuning* de `XGBoost`;
- en `neural` Fase 4: detector + optimizador `XGBoost` conjunto, heredando la mejor arquitectura previa.

Objetivo optimizado Cada trial corre las tres semillas y agrega sus registros por mes calendario. El score de ranking es

$$\text{score} = \text{median_month}(\text{PR-AUC}) - 0,05 \cdot \text{action_cost} - 0,10 \cdot \text{stability_penalty},$$

donde

$$\text{action_cost} = \min\left(\frac{n_{\text{actions}}}{n_{\text{months}}}, 1\right), \quad \text{stability_penalty} = \text{std}(\text{PR-AUC}_{\text{monthly}}).$$

Por diseño, el sweep no premia sólo la mejor PR-AUC. También penaliza estrategias inestables o con demasiadas adaptaciones.

Cierre estadístico y promoción Al final, el runner recompone `leaderboard_dev.csv`, `leaderboard_holdout.csv`, `monthly_metrics.csv`, `param_importances.csv`, `pareto.csv`, `pairwise_stats.csv` y `winner_config.json`. El gate de promoción usa Wilcoxon unilateral + *bootstrap* BCa sobre meses de holdout y exige:

- $p < 0,05/3$ tras Bonferroni;
- límite inferior del intervalo de confianza > 0 ;
- ganancia mediana en PR-AUC mensual $> 0,01$.

El `winner_config.json` sólo se genera para el estudio `neural` y queda marcado como `eligible_for_promotion` si supera ese gate frente a `cumulative`.

CLI de experimentos `src/experiments/drift_sweep.py` expone el sweep fuera de Streamlit. Sus flags son `-study`, `-phase`, `-resume-run-id`, `-feature-export` y `-balance-mode`. La salida es un JSON con `run_id`, `manifest_path` y `artifacts_dir`.

5.17. Validación mediante pruebas

La cobertura ya no vive sólo en `tests/test_drift_detection_app.py`. Para *Neural drift*, la evidencia principal está en dos suites específicas.

`tests/test_neural_drift_app.py` Esta suite valida:

- alineamiento temporal de ventanas y horizonte: `test_build_window_dataset_aligns_prediction_horizon`, `test_split_window_dataset_fixed_dates_uses_2018_base_and_2019_stream`;
- sensibilidad del canal embedding: `test_embedding_drift_score_increases_under_regime_shift`;
- presets del detector: `test_drift_monitor_profiles_define_moderate_and_sensitive_presets`, `test_detector_sensitivity_presets_span_conservative_to_very_sensitive`;
- rama neuronal paralela de XGBoost: `test_train_xgboost_model_attaches_parallel_neural_branch`, `test_build_channel_scores_combines_xgb_and_parallel_neural_signals`;
- optimizaciones de *fine-tuning*: `test_fine_tune_artifact_uses_dynamic_window_and_recency_weights`, `test_xgb_fine_tune_selection_metric_changes_candidate_choice`, `test_run_backtest_pipeline_record`;
- attention del predictor y del detector: `test_temporal_attention_monitor_returns_normalized_attention_weights`, `test_backtest_with_temporal_attention_detector_returns_detector_attention_outputs`, `test_attention_model_detects_drift_and_reports_attention_shift`;

- comparación none vs. smote: `test_run_backtest_pipeline_compares_none_and_smote_results`;
- persistencia y reanudación: `test_run_backtest_with_checkpoints_persists_manifest_and_artifacts`, `test_run_backtest_with_checkpoints_resumes_only_pending_experiments`.

`tests/test_neural_drift_experiments.py` Esta suite cubre el nivel de sweep experimental:

- validez del protocolo temporal fijo: `test_validate_experiment_dataset_accepts_full_protocol`;
- métricas mensuales y score de ranking: `test_monthly_metrics_and_score_use_calendar_months`;
- gate estadístico: `test_pairwise_stat_row_passes_gate_for_consistent_holdout_gain`;
- persistencia y resume del runner: `test_run_experiment_plan_persists_and_resumes_baseline_only`;
- CLI: `test_drift_sweep_run_cli_resolves_dataset_and_returns_paths`.

Finalmente, `tests/test_drift_detection_app.py` sigue siendo relevante porque valida la generación del dataset upstream y los baselines `cumulative/adaptive_adwin` que el runner experimental reutiliza. En conjunto, estas pruebas muestran que la documentación de *Neural drift* describe el comportamiento realmente implementado en el repositorio, no una arquitectura aspiracional.

6. Introducción

Este documento detalla el proceso de ingeniería de características (*feature engineering*) utilizado para agrupar conductores (patentes) en función de su comportamiento en la autopista. El objetivo es transformar los registros brutos de flujos vehiculares en un vector de características único por patente.

7. Preprocesamiento de Datos

Antes de calcular las variables, los datos de flujo (F) pasan por una etapa de limpieza:

1. **Normalización de Patentes:** Se eliminan caracteres no alfanuméricos y se estandariza a mayúsculas. Patentes inválidas (ej. "NULL", "NAN") o con longitud fuera del rango $[5, 6]$ son descartadas.
2. **Filtrado de Carriles:** Se consideran únicamente los carriles 1, 2 y 3.
3. **Eliminación de Duplicados:** Se eliminan registros duplicados considerando la tupla (*Patente*, *Portico*, *Carril*).
4. **Tratamiento de Outliers (Velocidad):** Se aplica *Winsorization* a la velocidad por cada grupo de (*Portico*, *Carril*), limitando los valores extremos a los percentiles 1% y 99%.

8. Cálculo de Variables (Nivel Evento)

Para cada evento i (un paso de un vehículo por un pórtico), se calculan métricas intermedias. Sea v_i la velocidad del evento i en el instante t_i .

8.1. Headway (h_i)

El *headway* se calcula como la diferencia de tiempo con el vehículo precedente ($i-1$) en el **mismo carril y pórtico**. Se requiere que los datos estén ordenados temporalmente.

$$h_i = t_i - t_{i-1} \quad (1)$$

Si $h_i > 60$ segundos, se considera que no hay interacción cercana y el valor se descarta (NaN) para efectos de cálculo de conflicto, aunque puede contabilizarse en promedios si se desea.

8.2. Velocidad Relativa Indexada ($v_{rel,i}$)

Para normalizar el comportamiento según las condiciones del tráfico, se compara la velocidad del vehículo con la velocidad promedio del flujo en ese pórtico durante un intervalo de 5 minutos, denotada como $\bar{V}_{5min}$.

$$v_{rel,i} = \frac{v_i}{\bar{V}_{5min}} \quad (2)$$

- $v_{rel,i} > 1$: Conductor más rápido que el flujo promedio.
- $v_{rel,i} < 1$: Conductor más lento que el flujo promedio.

8.3. Conflicto y TTC (Time To Collision)

Se evalúa el riesgo de colisión con el vehículo precedente. El TTC_i se calcula solo si el vehículo i viaja más rápido que el vehículo anterior ($v_i > v_{i-1}$) y el headway es válido.

$$TTC_i = \frac{h_i \cdot v_i}{v_i - v_{i-1}} \quad (3)$$

Se define un evento de **conflicto** (c_i) si el TTC_i es menor a un umbral específico por pórtico (TTC_{max}), definido empíricamente.

$$c_i = \begin{cases} 1 & \text{si } TTC_i < TTC_{max} \\ 0 & \text{si } TTC_i \geq TTC_{max} \text{ o } v_i \leq v_{i-1} \end{cases} \quad (4)$$

9. Agregación por Patante

El objetivo es obtener un vector único por conductor P . El proceso de agregación puede realizarse de dos formas: Global o Ponderada Mensual.

Sea N_P el número total de pasadas registradas para la patente P .

9.1. Variables Agregadas

9.1.1. Velocidad Promedio ($\bar{v}_P$)

$$\bar{v}_P = \frac{1}{N_P} \sum_{i \in P} v_i \quad (5)$$

9.1.2. Velocidad Relativa Promedio ($\bar{v}_{rel,P}$)

$$\bar{v}_{rel,P} = \frac{1}{N_{rel}} \sum_{i \in P} v_{rel,i} \quad (6)$$

Donde N_{rel} es el número de eventos con velocidad relativa válida.

9.1.3. Headway Promedio ($\bar{h}_P$)

$$\bar{h}_P = \frac{1}{N_h} \sum_{i \in P, h_i \leq 60} h_i \quad (7)$$

9.1.4. Tasa de Conflicto (CR_P)

Proporción de eventos con headway válido que resultaron en conflicto.

$$CR_P = \frac{\sum_{i \in P} c_i}{N_h} \quad (8)$$

9.1.5. Uso de Carriles ($Lane_k$)

Proporción de uso del carril $k \in \{1, 2, 3\}$.

$$Lane_k = \frac{\text{Conteo de pasadas en carril } k}{N_P} \quad (9)$$

9.1.6. Tasa de Cambio de Pista (LCR_P)

Se estima contando cuántas veces el vehículo cambió de carril entre pórticos consecutivos. Sea L_j el carril en el evento j (ordenado por tiempo).

$$Cambios = \sum_{j=2}^{N_P} \mathbb{I}(L_j \neq L_{j-1}) \quad (10)$$

$$LCR_P = \frac{Cambios}{N_P - 1} \quad (11)$$

Nota: Esta métrica es aproximada ya que asume continuidad entre pórticos registrados.

9.2. Estrategias de Consolidación

9.2.1. 1. Agregación Global (Default)

Se aplican las fórmulas anteriores sobre **todos** los registros históricos de la patente P como un solo conjunto.

9.2.2. 2. Ponderación Mensual

Se calculan primero los promedios $X_{P,m}$ para cada mes m , y luego se ponderan por la cantidad de viajes en ese mes ($N_{P,m}$).

Sea X una variable cualquiera (ej. Velocidad, Conflict Rate). El valor final ponderado es:

$$X_{P,final} = \frac{\sum_m (X_{P,m} \cdot N_{P,m})}{\sum_m N_{P,m}} \quad (12)$$

Esto asegura que los meses con mayor actividad tengan mayor influencia, pero permite calcular métricas mensuales intermedias si se requiere análisis temporal.

9.2.3. 3. Actividad Temporal

Además de las variables conductuales, el módulo persiste métricas de actividad histórica por patente: `n_days_active`, `n_weeks_active`, `n_months_active` y `n_years_active`. Los meses se cuentan como pares año-mes calendario, por lo que 2024-01 y 2025-01 son meses distintos. Estas columnas se usan tanto para definir conductores frecuentes como para seleccionar la muestra de patentes prevalentes en GMM dinámico.

10. Definición de Conductores Frecuentes

Para el entrenamiento del modelo de clustering, se separan los usuarios en:

- **Frecuentes:** Usuarios con suficiente historia para caracterizar su comportamiento. Criterios típicos:
 - $N_P \geq 20$ pasadas totales.
 - Días activos ≥ 5 .
 - Meses activos según `n_months_active` cuando se requiere persistencia temporal mínima.
- **Infrecuentes:** Usuarios con pocos datos. No se usan para entrenar, pero pueden ser asignados a un cluster posteriormente usando el modelo entrenado (o quedar como "Desconocido").

11. Implementación en Python

La lógica de cálculo descrita anteriormente está implementada principalmente en el módulo `src/clustering.py`. A continuación se detalla el flujo de ejecución de las funciones clave para fines de auditoría.

11.1. Limpieza de Datos: `clean_flujos_for_clustering`

Esta función recibe el DataFrame de flujos brutos y realiza la preparación inicial:

1. **Validación de Columnas:** Se asegura que existan las columnas esenciales (timestamp, velocidad, pórtillo, carril, patente).
2. **Limpieza de Patentes:** Se normalizan las patentes (alfanuméricos, mayúsculas) y se filtran aquellas con longitud inválida (fuera del rango [5, 6]).

3. **Filtrado Básico:** Se eliminan filas con datos nulos en campos críticos y se conservan solo los carriles 1, 2 y 3.
4. **Deduplicación:** Se eliminan duplicados exactos de la tupla (Patente, Pórtico, Carril, Timestamp). Si existen duplicados con diferentes velocidades, se prioriza determinísticamente (ordenamiento merge sort preservando el primero).
5. **Winsorization:** Se limitan los valores extremos de velocidad por grupo (Pórtico, Carril) usando los percentiles definidos (default 1 % y 99 %).

11.2. Cálculo de Features: Clusterization

Esta es la función principal que orquesta el cálculo de variables.

11.2.1. 1. Métricas Simples y Pre-cálculo

Se calculan métricas que no requieren ordenamiento temporal complejo:

- **Total de Pasadas (total_passes):** Conteo simple de registros por patente.
- **Velocidad Promedio (avg_speed_kmh):** Promedio aritmético de la velocidad.
- **Uso de Carril:** Se construye una tabla pivote para contar pasadas por carril y dividir por el total.

11.2.2. 2. Velocidad Relativa

- Se calcula la velocidad promedio del flujo en ventanas de 5 minutos por pórtico (`interval_speed_mean`).
- Se cruza este promedio con cada evento para calcular $\text{relative_speed} = v_i / \bar{V}_{5min}$.

11.2.3. 3. Métricas de Interacción (Headway, Conflicto)

Esta etapa es crítica y sensitiva al orden de los datos:

1. **Ordenamiento:** Los datos se ordenan estrictamente por Pórtico, Carril y Timestamp.
2. **Operaciones Vectorizadas (Shift):** Se crean columnas desplazadas (`shift(1)`) para comparar el evento actual con el vehículo inmediatamente anterior en el mismo carril/pórtico.
3. **Cálculo de Headway:** Diferencia de tiempo entre el evento actual y el anterior. Se descartan (NaN) los headways mayores a `max_headway_s` (60s).
4. **Cálculo de Conflicto:**
 - Se calcula $TTC = \frac{\text{Headway} \cdot v_{actual}}{v_{actual} - v_{anterior}}$.
 - Solo es válido si $v_{actual} > v_{anterior}$.
 - Se marca conflicto si $TTC < TTC_{max}$ (donde TTC_{max} depende del pórtico).
5. **Agregación:** Se suman y cuentan los eventos válidos por patente para obtener promedios.

11.2.4. 4. Cambios de Pista (Lane Changes)

- Se ordenan los datos por Patente y Timestamp.
- Se compara el carril actual con el anterior. Si son diferentes (y es la misma patente), se cuenta como cambio.
- La tasa se normaliza por $(N_P - 1)$.

11.2.5. 5. Ponderación (Si aplica)

Si se activa `monthly_weighting`, las métricas se calculan primero agrupando por Patente-Mes, y luego se hace un promedio ponderado por la cantidad de viajes del mes para obtener el valor final de la patente.

12. Alcance y metodología

Este informe documenta y audita el pipeline GNN implementado en el repositorio, con foco en:

- El flujo *tab a tab* de la aplicación Streamlit (`src/graph_builder_app.py`).
- La construcción del grafo, el modelo GNN, el entrenamiento y la evaluación (`src/graph.py`, `src/gat_model.py`, `src/gnn_main.py`, `src/train_pretrain.py`).
- Los mecanismos de balanceo: GraphSMOTE e ImGAGN (`src/graphsmote.py`, `src/imgagn.py`).
- La búsqueda de hiperparámetros (Optuna) y el control de métricas.

La auditoría incluye verificación de mejores prácticas, riesgos de fuga de información, uso de memoria/cálculo, y consistencia con las opciones de configuración (`src/config.py`).

13. Mapa general del pipeline

Resumen conceptual:

1. **Datos base:** pórticos, flujos y accidentes (`src/utils.py`).
2. **Feature engineering:** snapshot/flow (`src/snapshot_pipeline.py`, `src/features.py`).
3. **Feature selection:** selección manual/guiada (UI) y persistencia.
4. **Grafo:** nodos pm, aristas temporal/spatial/st_fwd y atributos (`src/graph.py`).
5. **Entrenamiento:** HeteroGAT + TemporalAggregator (opcional) (`src/gat_model.py`, `src/temporal_head.py`).
6. **Balanceo:** GraphSMOTE (embedding-space + $z \rightarrow x$ + edge-gen) y/o ImGAGN (adversarial) (`src/graphsmote.py`, `src/imgagn.py`).
7. **Evaluación:** métricas (F1, AUC, AUPRC, MCC), umbrales y calibración (`src/gnn_main.py`).
8. **Artefactos:** modelos, hparams JSON, grafos aumentados, historial (`Resultados/`, `gnn_history.jsonl`).

14. Pipeline *tab a tab* (Streamlit Graph Builder)

El flujo UI se implementa en `src/graph_builder_app.py` y organiza el trabajo en tabs.

14.1. Eventos

Función UI: `_render_events_tab`.

Objetivo: cargar y validar accidentes, normalizar columnas, limpiar y preparar el dataset de eventos.

- Carga de accidentes desde CSV/archivos seleccionados, persistencia en `st.session_state`.
- Uso de utilidades en `src/utils.py` para normalizar columnas y fechas.
- Soporte para generar reportes de *Puntos Negros* (posteriormente usados en *Graph*).

Implicación: este tab define el universo de accidentes, que afectará etiquetas futuras y cortes temporales.

14.2. Feature Engineering

Función UI: `_render_feature_engineering`.

Objetivo: generar o cargar features por nodo `pm`.

- Modos principales: *Snapshot Features* y *Flow 5 min* (ver `src/snapshot_pipeline.py`).
- Generación de features con `compute_pm_features` (`src/features.py`) y/o `SnapshotFeatureBuilder`.
- Salida persistida en DuckDB o CSV; sincronización de rango temporal y tramo.

Implicación: afecta dimensionalidad de nodos y distribución de atributos. La resolución temporal se gobierna con `DT_MINUTES` y parámetros en `src/config.py`.

14.3. Feature Explorer

Función UI: `render_feature_explorer`.

Objetivo: inspección visual, estadísticas y outliers de variables de entrada. **Implicación:** diagnóstico de calidad de features antes de construir el grafo.

14.4. Feature Selection

Función UI: `_render_feature_selection_tab`.

Objetivo: seleccionar subconjuntos de variables para entrenamiento, guardar listas y metadatos.

Implicación: cambia el espacio de entrada del GNN y la dimensionalidad de `pm.x`.

14.5. Graph

Funciones UI: `_render_create_graph`, `_render_load_graph`, `_render_in_memory_graph`.

Objetivo: construir o cargar el grafo heterogéneo de pórticos.

- Selección de tramo (manual o por puntos negros).
- Construcción de nodos `pm` (pórtico-minuto) con features normalizados.
- Generación de aristas temporales/espaciales y atributos de arista (ver Sección 15).

- Creación de `train/val/test_mask` con el split temporal purgado implementado por `_compute_temporal_split_mask`.
- Normalización estricta con estadísticos estimados sólo sobre `train_mask` purgado.
- Guardado en `Resultados/highway_graph_*.pt` y en memoria (`st.session_state`) con metadata autosuficiente y hash semántico del grafo.

14.6. Visual Graph

Función UI: `render_visual_graph_tab`.

Objetivo: visualizar topología, grados, y subgrafos para QA.

14.7. Network

Función UI: `_render_network_tab`.

Objetivo: configurar arquitectura y parámetros del modelo antes de entrenar. **Implicación:** fija opciones como `hidden_channels`, `num_heads`, `dropout`, `num_layers`, agregaciones `aggr1/aggr2` y checkpointing.

14.8. Optuna

Función UI: `_render_optuna_tab`.

Objetivo: búsqueda de hiperparámetros con Optuna (ver Sección 20). **Implicación:** almacena `optuna_hyperparams_*.csv` y gobierna la selección automática en entrenamiento.

14.9. Balance

Función UI: `_render_balance_tab`.

Objetivo: generar grafos balanceados con GraphSMOTE o ImGAGN.

- GraphSMOTE: requiere modelo entrenado para embeddings y decodificadores $z \rightarrow x$.
- ImGAGN: entrena generador y discriminador adversarial y crea nodos sintéticos.

Implicación: genera grafos con `is_synthetic`, actualiza `train_mask`, y puede cambiar el balance real.

14.10. Training

Función UI: `_render_training_tab`.

Objetivo: entrenar el GNN con opción de grafo balanceado, early stopping y evaluación automática.

Backend: `src/gnn_main.py::run_gat_training` y `src/train_pretrain.py::train_minibatch`.

14.11. Evaluación

Función UI: `_render_evaluation_tab`.

Objetivo: cargar modelos, evaluar métricas, aplicar umbral y calibración opcional. **Backend:** `src/gnn_main.py::test` y utilidades de exportación.

14.12. History

Función UI: `_render_history_tab`.

Objetivo: registro de experimentos y trazabilidad (`Resultados/gnn_history.jsonl`).

14.13. Experiments

Función UI: `_render_gnn_experiments_tab`.

Objetivo: ejecutar lotes experimentales, exportar resultados, e importar ZIPs.

15. Modelo de datos del grafo

Nodos: `pm` (pórtico-minuto).

Targets: `pm.y` (etiqueta binaria), `pm.is_accident_pm` (accidente directo).

Máscaras: `train_mask/val_mask/test_mask/sequence_mask`.

Metadata: el builder Streamlit persiste `metadata`, `edge_config`, `normalization`, `sequence_index`, `sequence_config` y `graph_hash` dentro del payload `.pt`.

15.1. Aristas y atributos

Aristas principales (`src/graph_builder_app.py`; `src/graph.py` queda como builder base CLI/de referencia):

- **temporal:** $P_{i,t} \rightarrow P_{i,t+1}$ (misma ubicación, siguiente tiempo).
- **spatial:** $P_{i,t} \rightarrow P_{i+1,t}$ (siguiente pórtico, mismo tiempo).
- **st_fwd** (opcional): $P_{i,t} \rightarrow P_{i+1,t+1}$.
- **spatial_back** (opcional): inversa de **spatial**.

Atributos de arista: el primer bloque es la diferencia destino-origen (Δx) sobre el subconjunto de features delta seleccionado por el usuario después de normalizar los nodos. Para Snapshot, el default compacto de `edge_delta_features` usa `speed_mean`, `speed_harmonic`, `flow_total`, `slow_pct` y `speed_std` cuando existen; si el usuario selecciona otras columnas, se respetan. El grupo Snapshot `spatial_gradients` no se activa por default, aunque sigue disponible como selección explícita. El segundo bloque contiene canales físicos opcionales: distancia, gradiente de flujo, gradiente de densidad, gradiente de velocidad, velocidad de onda de choque, indicador de choque y, para Snapshot, gradiente de `slow_pct`.

`_build_raw_global_edge_features` separa dos familias. En `classic` (*Flow 5 min*) conserva $q = \text{flujo_total}$, $k = \text{densidad_total}$, $v = v_{\text{armonica}}$ y $\pi = \text{prop_pesados}$, con fallbacks por categoría. En `snapshot` usa $q = \text{flow_total}$, $v = \text{speed_harmonic}$ con fallback a `speed_mean`, $k = q / \max(v, \epsilon)$ como proxy de densidad, $\pi = \text{mix_cat_2}$ con fallback `flow_cat_2/flow_total`, y $s = \text{slow_pct}$ con fallback cero. **spatial** calcula extras entre origen y destino del mismo timestamp; **spatial_back** invierte origen/destino y recalcula gradientes direccionales. **temporal** rellena ceros en canales físicos. **st_fwd** calcula gradientes espacio-temporales entre $P_{i,t}$ y $P_{i+1,t+1}$, preservando la propagación experimental desde el pórtico futuro.

15.2. Etiquetado temporal y split purgado

Ventana de etiqueta: las etiquetas snapshot usan

$$\text{intervalo positivo} = (t + \gamma, t + \gamma + H],$$

donde γ es la banda de guarda y H es el horizonte futuro. Para features snapshot, `_label_lookahead_minutes` devuelve $\gamma + H$. Para flow features, devuelve el paso de features Δt . Ese valor es el largo de embargo usado por el split.

Cortes temporales: el builder calcula los cortes desde la línea temporal completa de features, `df_pm["ts_min"].unique()`, no desde timestamps de accidentes. Esto evita condicionar el split al target positivo y preserva periodos negativos.

Máscaras purgadas: para los cortes c_{train} y c_{val} , las máscaras se asignan como:

$$\begin{aligned}\text{train}(t) &\Leftrightarrow t + L \leq c_{train}, \\ \text{val}(t) &\Leftrightarrow c_{train} < t \wedge t + L \leq c_{val}, \\ \text{test}(t) &\Leftrightarrow t > c_{val},\end{aligned}$$

donde L es `label_lookahead_minutes`. Luego las máscaras se intersectan con `sequence_mask`. Si train, validación o test queda vacío después del embargo, la construcción en Streamlit se detiene y pide ampliar el rango temporal o ajustar los ratios de split.

Re-split de grafos existentes: Training, Optuna, Ray Tune y Experiments llaman `_apply_temporal_split_t`. Si la metadata contiene `metadata["labeling"]["label_lookahead_minutes"]`, se reutiliza el mismo embargo. Los grafos legacy sin esa metadata usan fallback con embargo 0 y muestran una advertencia; deben reconstruirse para evaluación estricta tipo paper.

15.3. Normalización y payload reproducible

Normalización: los features de nodo se estandarizan como

$$x'_{u,f} = \frac{x_{u,f} - \mu_f}{\sigma_f},$$

con μ_f y σ_f calculados sólo sobre `train_mask` purgado. Los canales físicos de arista no se derivan de la matriz normalizada; usan la copia cruda capturada inmediatamente antes de normalizar.

Metadata guardada: los artefactos de grafo incluyen `schema_version`, `feature_config`, tramo seleccionado, franja horaria, fuente de eventos, `dt`, configuración de etiquetas, `label_lookahead_minutes`, cortes y conteos por split, `seed`, `edge_config`, columnas de features, resumen de normalización y `graph_hash`. `edge_config` registra `raw_feature_family`, `edge_attr_schema` y `edge_feature_sources` para auditar el mapeo $q/k/v/\pi/s$. El hash semántico se produce con `_compute_graph_semantic_hash` a partir de features, etiquetas, máscaras, índices y atributos de arista, `pm_index`, metadata de split, configuración de aristas y metadata general.

16. Arquitectura del modelo GNN

Modelo base: `src/gat_model.py::HeteroGAT`.

- **Capas:** `num_layers` bloques `HeteroConv` con `GATConvSaveAlpha`.
- **Agregación:** `aggr1` en la primera capa, `aggr2` en capas siguientes.
- **Atenciones:** capturadas cuando `XAI=1`, usadas para regularización $L2$.
- **Edge attributes:** se validan por batch; si faltan o desalinean, se rellenan con ceros.
- **Checkpointing:** `use_checkpointing` reduce memoria con recomputación.

16.1. Temporal head (opcional)

Clase: `src/temporal_head.py::TemporalAggregator`.

Idea: GRU sobre secuencias de embeddings recientes; mantiene cache de embeddings para nodos no presentes en el batch, evitando fuga de gradientes.

16.2. Registro de variantes GNN (encoder + temporal + aristas)

El pipeline ahora incorpora un **registro de variantes** controlado por `src/config.py` (`GNN_VARIANT`, `GNN_TEMPORAL_CACHE_STRATEGY`) y materializado en `src/gnn_main.py` mediante las funciones `_build_gnn_model`, `_build_temporal_head_for_variant` y `_prime_temporal_cache_if_needed`. Además, la UI en `src/graph_builder_app.py` expone una guía resumida en los tabs *Network*, *Training*, *Optuna* y *Evaluación*. La idea de diseño es separar:

- **Encoder espacial:** GAT heterogéneo base, GAT con codificador de aristas, o variante edge-aware.
- **Head temporal:** snapshot (sin secuencia), GRU, Transformer temporal o attention pooling.
- **Protocolo de cache:** incremental por batch o `global_epoch` (forward global por época para secuencias consistentes).

16.2.1. Idea general de comparabilidad

Todas las variantes comparten el mismo `SequenceIndex` (construido en `src/snapshot_sequences.py` a partir de `sequence_rows/target_rows`), el mismo grafo y los mismos splits temporales. Esto permite comparar arquitecturas sin cambiar la definición de etiqueta ni el horizonte de predicción.

Etiquetado “5 minutos antes” La lógica de etiquetado usa:

$$\text{intervalo positivo} = (t + \gamma, t + \gamma + H]$$

donde γ es `GUARD_BAND_MINUTES` y H es `HORIZON_MINUTES`. Para una configuración estricta de “5 minutos antes”, se recomienda:

- `GUARD_BAND_MINUTES` = 5
- `HORIZON_MINUTES` = 5 (o 10–20 para riesgo en ventana)

en `src/config.py`.

16.2.2. Variantes implementadas

Implementación: `src/gat_model.py`, `src/temporal_head.py`, `src/temporal_heads.py`, `src/gnn_main.py`.

Selección: mediante el string `GNN_VARIANT` (normalizado en `src/gnn_main.py::_normalize_gnn_variant`).

1. `gat_snapshot` (baseline): HeteroGAT sin head temporal. Clasifica con el embedding del snapshot actual.
2. `gat_gru`: HeteroGAT + TemporalGRUHead (`src/temporal_heads.py`).
3. `gat_transformer`: HeteroGAT + TemporalTransformerHead (self-attention con positional embedding aprendible).
4. `gat_attnpool`: HeteroGAT + TemporalAttnPoolHead (ablation ligera y barata computacionalmente).
5. `gat_edge_mlp`: HeteroGATWithEdgeEncoder (MLP por tipo de arista sobre `edge_attr`) sin head temporal.

6. `gat_edge_mlp_gru` y `gat_edge_mlp_transformer`: combinan codificador de aristas + head temporal.
7. `gnn_edge_aware`: `HeteroEdgeAware` (usa `TransformerConv`) sin head temporal.
8. `gnn_edge_aware_gru` y `gnn_edge_aware_transformer`: variante edge-aware con modelado temporal.

16.2.3. Codificación de aristas (`edge_attr`)

La variante `HeteroGATWithEdgeEncoder` agrega un MLP de aristas (`EdgeAttrEncoder`) antes de la atención. Motivación:

- los deltas Δx pueden tener escalas heterogéneas;
- un mapeo aprendido puede inducir una geometría más estable para la atención;
- permite estudiar de forma separada el efecto de “mejor `edge_attr`” vs. “mejor head temporal”.

16.2.4. Heads temporales y ensamblado de secuencias

Los heads temporales reutilizan la infraestructura de `TemporalAggregator`:

- `update_cache(...)` para cache global o parcial de embeddings.
- `_build_sequence_batch(...)` para reconstruir tensores $[B, L, D]$ usando `sequence_rows`.

Esto permite cambiar el operador temporal (GRU/Transformer/Attention Pooling) sin cambiar el contrato del entrenamiento en `src/train_pretrain.py` ni la firma de `forward`.

16.2.5. Estrategias de cache temporal (punto crítico)

- `incremental`: cache por batch (histórico, menor costo, más dependiente del sampler).
- `global_epoch`: priming por época con forward global (`compute_epoch_embeddings`), luego training/eval usan cache consistente.

Recomendación para artículo/ciencia: usar `global_epoch` para que las secuencias temporales tengan la misma referencia de embeddings dentro de una época.

16.2.6. Matriz experimental recomendada

1. `gat_snapshot` (baseline espacial)
2. `gat_gru`
3. `gat_transformer`
4. `gat_edge_mlp_gru`
5. `gnn_edge_aware_gru` (o `gnn_edge_aware_transformer`)

Mantener semilla, split temporal, `SequenceIndex` y protocolo de evaluación constantes.

16.2.7. Métricas para eventos raros

Para comparación científica y operativa, priorizar:

- **PR-AUC / AUPRC**: métrica principal en desbalance severo.
- **ROC-AUC**: referencia secundaria.
- **Recall@FAR/FPR objetivo**: útil para restricciones operacionales.
- **MCC**: robusta cuando hay fuerte asimetría de clases.

16.2.8. Trazabilidad de variantes

El entrenamiento y HPO guardan `gnn_variant` y `variant_tag` en sidecars de hiperparámetros, checkpoints y logs estructurados (`Resultados/gat_model_BEST*.pt`, `_hparams.json`, Optuna CSVs). Esto evita mezclar checkpoints de arquitecturas incompatibles.

17. Entrenamiento y optimización

Punto de entrada: `src/gnn_main.py::run_gat_training`.

17.1. Flujo del entrenamiento

1. **Dispositivo**: `get_auto_device()` (MPS > CUDA > CPU).
2. **Carga de datos**: `loaded_obj['data']`.
3. **ImGAGN automático** (si `AUTO_IMGAGN_PRETRAIN=1`).
4. **GraphSMOTE**: elección por usuario/flag y detección de sintéticos existentes.
5. **HPO**: selección de `optuna_hyperparams_*.csv` o búsqueda nueva.
6. **Modelo**: fábrica `_build_gnn_model` selecciona encoder/temporal head según `gnn_variant` y adjunta el head temporal cuando aplica.
7. **Decodificadores $z \rightarrow x$** : entrenados si GraphSMOTE activo (`train_z2x_decoders`).
8. **Edge generator**: `RelEdgeGen` para reconstrucción y aristas sintéticas.
9. **Pérdida**: `CrossEntropy` o `FocalLoss` (con/ sin pesos).
10. **Scheduler**: `OneCycleLR`.
11. **Loop**: entrenamiento por mini-batches, monitoreo de pérdida de validación, early stopping y guardado de mejor modelo.

17.2. Train-minibatch y regularización

Función: `src/train_pretrain.py::train_minibatch`.

- Clasificación sobre `pm` (primeros `batch_size` nodos supervisados del loader activo).
- Pérdida total:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_{edge} \mathcal{L}_{edge} + \lambda_{att} \mathcal{L}_{att} + \lambda_{simi} \mathcal{L}_{simi}.$$

- $\mathcal{L}_{edge}$: reconstrucción de aristas con muestreo negativo (si `edge_gen`).
- $\mathcal{L}_{att}$: $L2$ sobre atenciones cuando `XAI=1`.
- $\mathcal{L}_{simi}$: pérdida auxiliar label-aware usada sólo cuando el sampler `rl_top_p` está activo.
- Gradiente acumulado (`accumulation_steps=2`) y `grad_clip`.
- Soporte AMP (solo CUDA).

La implementación separa la pérdida usada para retropropagación de la pérdida reportada al usuario. Con acumulación de gradientes, `loss_for_backward = raw_loss / accumulation_steps`, pero `train_loss` a nivel de epoch y los eventos live por batch acumulan la `raw_loss` sin escalar. Por lo tanto, la curva de entrenamiento reportada estima el objetivo que se optimiza, no un artefacto del factor de acumulación.

17.3. Samplers de entrenamiento y RioGNN top-p RL

El entrenamiento admite cuatro familias de sampler mediante `train_sampler_mode`: `neighbor`, `cluster_gcn`, `graphsaint` y `rl_top_p`. `NeighborLoader` sigue siendo el baseline y no se elimina; `Cluster-GCN` particiona el grafo, `GraphSAINT` muestrea subgrafos estocásticos por nodos/aristas/caminatas, y `rl_top_p` filtra vecinos por relación y capa antes de entregar el batch al mismo modelo GNN.

La implementación `src/gnn_rl_sampler.py` adapta CARE-GNN/RioGNN al plano del loader:

- `LabelAwareSimilarityScorer`: MLP pequeño sobre `pm.x`; estima $p(y = 1 | x)$ y define similitud local $s(i, j) = 1 - |p_i - p_j|$.
- `RLTopPHeteroLoader`: expande el vecindario hop a hop; para cada relación toma top- p vecinos por similitud, respeta `num_neighbors` como cap de memoria y conserva `edge_attr`, `n_id`, `batch_size` y máscaras.
- `RioGNNThresholdController`: mantiene thresholds $p_{\ell, r}$ por capa ℓ y relación r , historial de recompensas, mejor threshold y convergencia.

RSRL tiene dos modos. `discrete` usa grilla recursiva de $\alpha = 10$ acciones, reduce el rango cuando la acción se estabiliza por `rl_switch_patience=3` y aplica backtracking por defecto. `continuous_actor` usa actor-critic PyTorch interno, sin dependencias externas, con salida acotada a `[rl_min_p, rl_max_p]`. Los valores por defecto son `rl_initial_p=0.5`, `rl_min_p=0.05`, `rl_max_p=1.0`, `rl_min_keep=1`, `rl_positive_only=True`, `rl_similarity_pretrain_epochs=3`, `rl_lambda_simi=2.0` y `rl_action_space=discrete`.

La recompensa se actualiza después de validación y antes de construir el loader de la siguiente época:

$$R_{\ell, r}^{(e)} = \frac{\text{AUPRC}_{val}^{(e)} - \text{AUPRC}_{val}^{(e-1)}}{\max(|\text{AUPRC}_{val}^{(e-1)}|, \epsilon)} + 0,25 \Delta \bar{s}_{\ell, r}^{(e)}.$$

Si validación no contiene dos clases, o AUPRC no es finita, el controlador registra el evento pero no actualiza thresholds. Las estadísticas de similitud se calculan sobre centros positivos de entrenamiento cuando existen; si no existen, usa todas las semillas supervisadas.

Los artefactos persistidos incluyen `sampler_impl=rl_top_p_rsrl`, `rl_sampler_state`, `rl_thresholds`, `rl_reward_history` y pesos relacionales auditables derivados de los thresholds. Esos pesos **no** se aplican al agregador GNN en esta versión; el cambio es deliberadamente un loader alternativo. La evaluación no cambia: `test()` conserva `eval_neighbors_mode` y mide validación/test con la prevalencia real.

El protocolo de ablación recomendado compara `neighbor`, `fixed_top_p`, `rl_top_p_discrete` y `rl_top_p_continuous`, cruzando cada variante con/sin `FocalLoss` y con/sin `GraphSMOTE`. Esto separa el efecto del filtro relacional aprendido del efecto de la pérdida y del aumento sintético.

17.4. Early stopping y checkpoint BEST por pérdida de validación

Política en la pestaña final *Training*: la selección del modelo queda controlada por la pérdida de validación, no por el objetivo de clasificación usado para reportar métricas o rankear Optuna. En la epoch e , `run_gat_training` llama `test(..., criterion=criterion)` sobre `val_mask`; si la pérdida de validación falta, el split está vacío, no es finita o no puede computarse, el entrenamiento aborta en vez de caer silenciosamente a una proxy métrica. Esta es una salvaguarda metodológica: early stopping es una regla de regularización y selección de modelo sobre riesgo empírico retenido, mientras que F1/F0.5/AUPRC/FAR/MCC son diagnósticos sensibles al umbral y a la prevalencia.

Para el conjunto de nodos de validación V , con logits $z_i = f_{\theta_e}(x_i, G)$, etiquetas y_i y criterio ℓ , la cantidad monitorizada es

$$\mathcal{L}_{val}^{(e)} = \frac{1}{Z_V} \sum_{i \in V} \ell(z_i, y_i).$$

Para `CrossEntropyLoss`, Z_V es el número de ejemplos válidos de validación, o la suma de sus pesos de clase cuando el criterio ponderado se usa. Para `FocalLoss`, la implementación acumula pérdidas focales por nodo y divide por el número de nodos evaluados. Para criterios desconocidos, `test` usa un fallback conservador con suma finita y denominador, evitando la práctica sesgada de promediar pérdidas de mini-batches que ya vienen promediadas.

El estado del monitor es:

$$b_0 = +\infty, \\ \text{isBest}_e \iff (b_{e-1} - \mathcal{L}_{val}^{(e)}) > \delta,$$

con b_{e-1} como la `best_val_loss` anterior y δ como `early_stop_min_delta`. Si `is_best_e` es verdadero, la paciencia se reinicia y se escribe el checkpoint `gat_model_BEST*.pt`. En caso contrario, `patience_counter` aumenta y el entrenamiento se detiene cuando `patience_counter` $\geq$ `early_stop_patience`. El sidecar registra `monitor_metric="val_loss"`, `monitor_mode="min"`, `best_val_loss`, `best_epoch` y las métricas diagnósticas observadas en esa misma epoch: F1, F0.5, AUPRC, AUC, MCC, FAR, accuracy y el umbral seleccionado. Estas métricas siguen siendo evidencia de auditoría de primer orden, pero ya no deciden cuándo parar ni qué checkpoint final recibe el nombre BEST.

La semántica de reanudación es deliberadamente conservadora. Si un checkpoint contiene `best_val_loss`, se restauran el monitor y el contador de paciencia. Si el checkpoint es anterior a este contrato y no contiene `best_val_loss`, se reanudan pesos y estado del optimizador, pero el monitor por pérdida de validación parte desde $+\infty$ con paciencia cero. Así se evita mezclar la regla antigua basada en objective metric con la nueva selección basada en pérdida.

El stream de eventos live, los checkpoints temporales y el evento `train_end` exponen `val_loss`, `best_val_loss`, `monitor_metric` y `monitor_mode`. La UI de Streamlit muestra entonces pérdida de

validación actual y mejor pérdida de validación, y el gráfico de pérdidas incluye `val_loss` junto a la pérdida de entrenamiento. Optuna/HPO no cambia: `src/gnn_main.py::objective` sigue rankeando trials con su `objective_metric` configurado; sólo el protocolo de entrenamiento final en *Graph Neural Networks > Training* usa pérdida de validación para early stopping y BEST.

18. GraphSMOTE: modificaciones sobre la arquitectura original

Módulo principal: `src/graphsmote.py`.

Puntos de integración: `src/gnn_main.py`, `src/train_pretrain.py`, `src/graph_builder_app.py`, `src/config.py`.

Referencia original: `papers/Modelos/GraphSMOTE.pdf`.

El paper de Zhao, Zhang y Wang define GraphSMOTE para clasificación semi-supervisada transductiva en un grafo homogéneo $G = \{V, A, F\}$, con una sola matriz de adyacencia A , features de nodo F , un extractor GraphSAGE, SMOTE en el embedding intermedio, un generador de aristas por producto bilineal y una pérdida conjunta $L_{node} + \lambda L_{edge}$. Para usar esta idea en el pipeline de riesgo vial fue necesario reespecificar la arquitectura. El problema local no es una red homogénea de citas o usuarios: es un grafo pórtico-minuto, temporal, heterogéneo en sus relaciones, con atributos físicos de arista, particiones cronológicas purgadas y una tasa positiva extremadamente baja.

18.1. Cambio 1: de grafo homogéneo a HeteroData pórtico-minuto

La formulación original opera sobre una matriz $A \in \mathbb{R}^{n \times n}$. La implementación local opera sobre `torch_geometric.data.HeteroData` y conserva tipos de arista $\text{pm} \rightarrow \text{pm}$: `spatial`, `temporal`, `st_fwd` y, cuando se habilita, `spatial_back`. Por ello `RelEdgeGen` no aprende un único S , sino una matriz S_r por relación:

$$s_r(u, v) = \mathbf{z}_u^\top S_r \mathbf{z}_v, \quad r = (\text{src}, \text{rel}, \text{dst}).$$

Esto preserva la semántica direccional de propagación temporal y espacial. Una arista temporal no tiene el mismo significado que una arista espacial aguas abajo; compartir un único generador mezclaría mecanismos causales distintos.

18.2. Cambio 2: extractor heterogéneo y temporal en lugar de GraphSAGE fijo

El paper usa un bloque GraphSAGE como extractor y otro como clasificador. En este repositorio GraphSMOTE se acopla al modelo GNN ya seleccionado por el registro de variantes: `HeteroGAT`, `HeteroGATWithEdgeEncoder`, `HeteroEdgeAware` y sus cabezas temporales GRU/Transformer/attention pooling. Los embeddings se extraen con `get_embeddings_minibatch` o `compute_train_embeddings`, pasando `edge_attr_dict` al forward para no descartar atributos de arista. Además, los embeddings se normalizan por nodo con ℓ_2 , lo que fija una geometría esférica para el k-NN y para los decodificadores.

18.3. Cambio 3: SMOTE sólo en entrenamiento y con embargo temporal

La interpolación se mantiene fiel a la ecuación de GraphSMOTE:

$$\mathbf{z}_{syn} = (1 - \alpha)\mathbf{z}_i + \alpha\mathbf{z}_j, \quad \alpha \sim U(0, 1),$$

donde j es un vecino minoritario de i . La modificación crítica es que el conjunto elegible se restringe a `train_mask`. En entrenamiento se usa `compute_train_embeddings`; en balance manual,

`run_graphsmote` también filtra el pool por `train_mask`. Así, nodos de validación o test nunca participan en la síntesis ni en el k-NN minoritario. Este ajuste es obligatorio porque las máscaras del grafo son cronológicas y purgadas por la ventana de etiqueta; usar embeddings de validación o test produciría fuga temporal.

La cantidad de sintéticos en el flujo automático se controla por `TARGET_POS_RATIO/target_pos_ratio`:

$$n_{new} = \max\{0, \lceil r N_{train} \rceil - n_+\},$$

donde r es el ratio objetivo, N_{train} el número de nodos de entrenamiento y n_+ los positivos reales de entrenamiento. En la UI manual, `_compute_n_syn_from_ratio` permite convertir un ratio final deseado en un número explícito de nodos.

18.4. Cambio 4: decodificadores $z \rightarrow x$ para volver al espacio de features

En el paper el clasificador consume directamente el conjunto aumentado de embeddings $\tilde{H}$. En el pipeline local el entrenamiento posterior espera un grafo PyG con `pm.x`; por tanto, no basta con generar $\mathbf{z}_{syn}$. Se incorporó `Z2XDecoders`, un MLP por tipo de nodo:

$$\mathbf{x}_{syn} = f_{\tau}(\mathbf{z}_{syn}),$$

donde τ es el tipo de nodo. `train_z2x_decoders` entrena estos decodificadores con SmoothL1 o MSE, early stopping y validación. Para evitar bucles de realimentación, excluye nodos marcados como `is_synthetic`. Además, agrega una regularización de consistencia tipo mixup sobre pares minoritarios: se interpola simultáneamente en z y x , y se exige al decoder reconstruir el punto interpolado. Esto alinea el entrenamiento del decoder con el uso real que tendrá durante GraphSMOTE.

18.5. Cambio 5: atributos de arista para aristas sintéticas

GraphSMOTE original predice sólo existencia de enlaces. El grafo vial usa `edge_attr` con diferencias de features y canales físicos como distancia, gradientes, velocidad de onda de choque e indicadores de congestión. Por eso se agregaron dos rutas:

- `RelEdgeAttrDecoder`: MLP por relación que aprende $[\mathbf{z}_{src} || \mathbf{z}_{dst}] \mapsto \mathbf{e}_{src, dst}$ usando aristas reales de entrenamiento.
- Fallback determinista: $\Delta x = x_{dst} - x_{src}$, restringido por `delta_feature_idx` si existe, truncado o rellenado para coincidir con la dimensionalidad de `edge_attr`.

Esto corrige el caso anterior en que los atributos sintéticos se rellenaban parcialmente con ceros cuando $\dim(\Delta x) < \dim(\text{edge_attr})$. La limpieza `_safe_clean_edges` aplica `remove_self_loops` y `coalesce` preservando `edge_attr` cuando es posible y eliminándolo sólo si queda desalineado.

18.6. Cambio 6: conectividad top-K en lugar de adyacencia densa o umbral global

El paper propone binarizar $E_{v',u}$ con un umbral η o usar aristas suaves. En grafos pórtico-minuto esto sería cuadrático y poco interpretable. La implementación predice vecinos por *top-K* y restringe candidatos reales mediante `GRAPHSMOTE_CONNECT`:

- `GRAPHSMOTE_CONNECT=1`: candidatos en `train_mask` real.
- `GRAPHSMOTE_CONNECT=0`: vecinos a un salto de accidentes reales de entrenamiento.

La dirección se controla con `BIDIRECTION`: si vale 1 se agregan aristas $\text{real} \leftrightarrow \text{sintético}$; si vale 0 se agregan sólo $\text{real} \rightarrow \text{sintético}$. El valor por defecto es dirigido, coherente con un pipeline predictivo donde no se desea que un nodo sintético inyecte evidencia artificial hacia nodos reales de validación o test.

18.7. Cambio 7: modos offline y online sin acumulación de sintéticos

El algoritmo del paper sintetiza durante la optimización. El repositorio agrega una política operacional:

- **Offline:** `augment_graph_offline_once` genera un grafo aumentado una vez y entrena sobre él.
- **Online:** `refresh_synthetics_online` vuelve siempre al grafo base real y regenera sintéticos cada `SMOTE EVERY_N_EPOCHS`.
- **None:** mantiene el entrenamiento sin aumentación.

El modo online no acumula generaciones previas; reemplaza el grafo aumentado para evitar crecimiento no controlado, deriva de distribución y aprendizaje sobre sintéticos producidos por versiones anteriores del modelo.

18.8. Cambio 8: pérdida, pesos de clase y diagnósticos

La pérdida conjunta se conserva conceptualmente, pero se implementa en mini-batches:

$$L = L_{cls} + \lambda_H L_H + \lambda_{edge} L_{edge} + \lambda_{att} L_{att}.$$

`train_minibatch` entrena `RelEdgeGen` con positivos reales del batch y negativos muestreados; `pretrain_edge_generator` lo preentrena con BCE y reporta AUC/AP de link prediction por relación. Cuando GraphSMOTE está activo se deshabilitan pesos de clase y `focal_alpha`; esto evita corregir dos veces el mismo desbalance. Sin GraphSMOTE, `CrossEntropyLoss` usa pesos templados $\sqrt{1/n_c}$, más estables que $1/n_c$ bajo prevalencias cercanas a cero.

18.9. Cambio 9: trazabilidad y garantías de auditoría

Los nodos sintéticos se agregan a `pm.x` y `pm.y`, entran sólo a `train_mask`, quedan fuera de `val_mask` y `test_mask`, se marcan con `is_synthetic=True` y con `is_accident_pm=False`. Esta última distinción es importante: un nodo sintético representa una muestra positiva para entrenamiento, no un accidente observado. El pipeline persiste decodificadores `z2x`, reportes `edge_gen_auc.json`, grafos aumentados, hiperparámetros con marca `_GraphSMOTE`, `graph_hash` y conteos de sintéticos en TensorBoard/Streamlit. En conjunto, estas modificaciones convierten GraphSMOTE de un método homogéneo de paper en un operador de aumentación reproducible, temporalmente seguro y compatible con el grafo vial heterogéneo del proyecto.

19. ImGAGN: generación adversarial

Módulo: `src/imgagn.py`.

Idea central: generar nodos sintéticos como combinaciones convexas de nodos minoritarios *de entrenamiento* y entrenar un discriminador adversarial.

19.1. Paso 1: homogenización

`to_homogeneous_if_needed` transforma `HeteroData` a grafo homogéneo (target `pm`), preservando `edge_attr` y `delta_feature_idx` si existen.

19.2. Paso 2: definir cantidad de sintéticos

$$\lambda_1 = \frac{n_{min} + n_g}{n_{maj}}$$
$$\Rightarrow n_g = \text{máx}(0, \lambda_1 n_{maj} - n_{min})$$

Se limita con `max_new_nodes` para evitar OOM.

19.3. Paso 3: generador

Clase: `ImGAGNGenerator`.

- Entrada: ruido $\mathbf{z} \sim \mathcal{N}(0, I)$.
- Salida: logits por nodo minoritario de entrenamiento.
- Pesos $T = \text{softmax}(G(z))$ y features sintéticos $X_g = TX_{min}$.
- Aristas: top-K conexiones a nodos minoritarios reales según T .

19.4. Paso 4: discriminador

Clase: `ImGAGNDiscriminator` (GCN + 2 cabezas).

- **Head real/fake** (BCE).
- **Head minority/majority** (BCE).
- Término de separación `_mm_separation` para alejar medias minoritaria y mayoritaria.

19.5. Paso 5: entrenamiento adversarial

Por cada epoch:

1. Muestrear ruido y construir sintéticos (sin gradientes).
2. Construir grafo aumentado y entrenar D por `d_steps` minibatches.
3. Recalcular X_g con gradientes y optimizar G para:

$$\mathcal{L}_G = \mathcal{L}_{rf} + \mathcal{L}_{mi} + \mathcal{L}_{di},$$

donde $\mathcal{L}_{rf}$ empuja a *real*, $\mathcal{L}_{mi}$ empuja a *minority*, y $\mathcal{L}_{di}$ acerca embeddings a centroides minoritarios.

Escalabilidad: usa `NeighborLoader` si está disponible; si no, full-batch.

19.6. Paso 6: grafo final

Se reconstruye el grafo aumentado con el generador final y se devuelven: `x_aug`, `edge_index_aug`, `edge_attr_aug`, y la porción de nodos generados.

20. Optuna (búsqueda de hiperparámetros)

Función: `src/gnn_main.py::objective`.

- **Objetivo:** maximizar F0.5 en validación (prioriza precisión).
- **Espacio:** `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`.
- **Optimizadores:** Adam/AdamW/RAdam (configurable).
- **Samplers:** `neighbor`, `cluster_gcn`, `graphsaint` y `rl_top_p`; para `rl_top_p` puede buscar `rl_action_space` entre `discrete` y `continuous_actor`.
- Con GraphSMOTE: busca `smote_k`, `target_pos_ratio`, `smote_every_n_epochs`, `lambda_edge`.
- Sin GraphSMOTE: FocalLoss con `focal_alpha`, `focal_gamma`.

Muestreo de seeds (train/val):

- *Resamplear seeds por epoch:* cambia el subset en cada epoch (mide robustez).
- *Fijar subset por trial:* mantiene el mismo subset durante el trial (reduce varianza).
- *Cargar todo el grafo en memoria (sin muestreo):* usa todos los nodos `train/val` (equivale a `sample_train_nodes=0` y `sample_val_nodes=0`); mayor uso de memoria.

21. Evaluación, calibración y artefactos

- **Evaluación:** `test` calcula pérdida de validación/test cuando recibe un criterio, además de F1, AUC, AUPRC, MCC y matriz de confusión.
- **Umbral:** `pick_tau_fbeta` busca umbral τ en validación.
- **Calibración:** Platt scaling opcional (`AUTOCALIBRATE_PROBS`).
- **Guardado:** modelos `gat_model_BEST*.pt`, `GraphSMOTE_embeddings_model*.pt`, `sidecar_hparams.json`; el checkpoint GNN final BEST se selecciona por mínima `val_loss`. Si se usa `rl_top_p`, el `sidecar` incluye estado, thresholds e historial de recompensa RL.
- **Hash del grafo:** los artefactos creados por el builder Streamlit guardan `metadata["graph_hash"]` y `graph_hash` de primer nivel, calculados con `_compute_graph_semantic_hash`.
- **Procedencia del split:** la metadata del grafo registra cortes, conteos por split, lookahead de etiqueta, uso de `sequence_mask`, cantidad de nodos purgados, features seleccionadas, configuración de aristas, resumen de normalización, fuente de eventos y seed.

22. Auditoría técnica y mejores prácticas

22.1. Fortalezas observadas

- El builder Streamlit ahora usa split temporal purgado desde la línea temporal completa de features, no desde timestamps positivos de accidentes.
- Los estadísticos de normalización se estiman sólo sobre nodos de entrenamiento purgados.
- Los canales físicos de aristas espaciales se calculan desde magnitudes de tráfico crudas antes de normalizar los nodos.
- Los payloads de grafo guardados incluyen metadata autosuficiente y hash semántico para reproducibilidad.
- GraphSMOTE en entrenamiento usa embeddings de **train** (`compute_train_embeddings`).
- `NeighborLoader`, `Cluster-GCN`, `GraphSAINT` y `rl_top_p` ofrecen baselines de muestreo comparables sin cambiar el protocolo de evaluación.
- Mecanismos de limpieza/*coalesce* para evitar inconsistencias en `edge_attr`.
- Guardado de hparams y `run_id` para reproducibilidad.
- El entrenamiento GNN final ahora usa pérdida de validación como monitor de early stopping y checkpoint BEST, separando minimización de riesgo empírico de métricas diagnósticas dependientes del umbral.

22.2. Riesgos o puntos críticos (actualizado)

- **Compatibilidad con grafos legacy:** archivos `.pt` antiguos sin `label_lookahead_minutes` siguen cargando, pero el re-split usa embargo 0 y muestra advertencia. Deben reconstruirse para evaluación temporal estricta.
- **Líneas temporales escasas:** el embargo estricto puede vaciar train, validación o test si el rango seleccionado es corto o si guard band/horizonte son grandes. En ese caso la app aborta la construcción del grafo.
- **Paridad de builders:** el contrato estricto de metadata y split purgado documentado aquí aplica a `src/graph_builder_app.py`. El builder CLI/de referencia en `src/graph.py` debe auditarse por separado antes de usarlo como generador de artefactos tipo paper.
- **Fuga en balanceo manual (mitigado):** `run_graphsmote` ahora restringe embeddings/pool minoritario a `train_mask`. Mantener verificación en auditorías.
- **Dependencia de BIDIRECTION:** dirección de aristas sintéticas afecta mensajes y puede sesgar la difusión de señales. Se mantiene sin cambios por ahora.
- **Acumulación de gradientes (ajustable):** `accumulation_steps` ahora es configurable (config/UI). Aún requiere tuning según memoria/hardware.
- **Comparabilidad de la pérdida de validación:** `val_loss` es interpretable sólo cuando split de validación, criterio, pesos de clase y variante de grafo se mantienen fijos. Las comparaciones entre corridas deben inspeccionar `graph_hash`, `gnn_variant`, `monitor_metric` y metadata del criterio en el sidecar.

- **Balanceo + pesos de clase:** se deshabilita pesos con GraphSMOTE/ImGAGN (correcto), pero requiere monitoreo de métricas en validación.
- **RioGNN top-p RL:** los pesos relacionales se guardan para auditoría pero no modifican el agregador; no deben interpretarse como importancia causal sin ablaciones.

22.3. Recomendaciones priorizadas (estado actual)

1. **Implementado:** usar la línea temporal completa de features y embargo purgado por ventana de etiqueta para `train/val/test` en el builder Streamlit.
2. **Implementado:** calcular z-score sólo con nodos train purgados y calcular canales físicos de arista desde valores crudos de tráfico.
3. **Implementado:** persistir metadata reproducible y hash semántico dentro del payload `.pt` del grafo.
4. **Implementado:** en `run_graphsmote` (balance manual) se limita a `train_mask` para evitar fuga potencial.
5. **Implementado:** exponer `accumulation_steps` en config/UI; ajustar según memoria disponible.
6. **Implementado:** agregar `rl_top_p` como sampler RL alternativo con AUPRC de validación como recompensa primaria, sin reemplazar `NeighborLoader`.
7. **Implementado:** seleccionar el checkpoint final BEST y disparar early stopping por mínima pérdida de validación; mantener F1/F0.5/AUPRC/FAR/MCC como métricas de reporte y auditoría.
8. **Pendiente/decisión:** no modificar `BIDIRECTION` por ahora; documentar el racional y evaluar impacto en AUPRC/F1 si se cambia en el futuro.
9. **Pendiente/decisión:** definir si `src/graph.py` debe migrarse al mismo contrato estricto de split y metadata que el builder Streamlit.
10. **Implementado:** en ImGAGN registrar `best_train_recall` junto a seed y config para trazabilidad.

23. Anexo: referencias a funciones y archivos

- UI principal: `src/graph_builder_app.py`.
- Helpers del builder Streamlit: `_label_lookahead_minutes`, `_compute_temporal_split_masks`, `_build_raw_global_edge_features`, `_compute_normalization_stats_from_train`, `_compute_graph_semanti`, `_apply_temporal_split_to_graph`.
- Construcción del grafo: `src/graph.py`.
- Modelo GNN: `src/gat_model.py`.
- Temporal head: `src/temporal_head.py`.
- Sampler RioGNN/CARE-GNN: `src/gnn_rl_sampler.py`.

- Entrenamiento/evaluación: `src/gnn_main.py`, `src/train_pretrain.py`; helpers clave del monitor: `_update_val_loss_monitor`, `_criterion_loss_sum_and_denominator` y `test(..., criterion=...)`.
- GraphSMOTE: `src/graphsmote.py`.
- ImGAGN: `src/imgagn.py`.
- Configuración: `src/config.py`.
- Tests: `tests/test_gnn_pipeline.py`, `tests/test_gnn_training.py`.

24. Implementación de MA-AIRL

24.1. Objetivo y alcance

Se implementó una versión funcional del algoritmo Multi-Agent Adversarial Inverse Reinforcement Learning (MA-AIRL) siguiendo el marco descrito en `src/MAIRL.pdf`. El objetivo es aprender, por cada clúster de conductores, una política estocástica y una función de recompensa que expliquen las demostraciones observadas en los datos agregados de clústeres, y luego exportar parámetros de *vTypes* para SUMO.

24.2. Arquitectura general

La implementación se dividió en dos capas:

- **Núcleo MA-AIRL** en `src/marl_core.py`: datos expertos, normalización, redes (política, recompensa y potencial), entrenamiento y exportación de parámetros.
- **Interfaz** en `src/multi_agent_rl_app.py`: carga de datos, selección de *features*, control de hiperparámetros, ejecución del entrenamiento y visualización de métricas.

24.3. Datos expertos y construcción del estado

Los datos de entrada son archivos CSV de clústeres con la columna `cluster_label`. Cada fila se interpreta como una muestra experta, con un estado s compuesto por columnas numéricas seleccionadas (por defecto: `avg_speed_kmh`, `avg_headway_s`, `lane_change_rate`). Se implementó:

- **Selección de features** desde la UI para definir el vector de estado.
- **Estandarización** con un `StandardScaler` propio (media y desviación por columna).
- **Separación por agente** agrupando por `cluster_label`.

24.4. Acciones expertas (proxy de SUMO)

MA-AIRL requiere pares (s, a) . Como el CSV no contiene acciones reales, se construyó un **proxy de acción** para cada muestra:

- Se mapean métricas a parámetros de car-following: `maxSpeed`, `accel`, `decel`, `minGap`, `sigma`, `impatience`.
- Se aplican transformaciones heurísticas y se recortan a los límites de `PARAM_BOUNDS`.
- Las acciones se normalizan a $[-1, 1]$ para entrenamiento estable.

Esto permite entrenar MA-AIRL aunque no existan trayectorias con acciones explícitas.

24.5. Modelos de MA-AIRL

Para cada agente (clúster) se instancian tres redes:

- **Política gaussiana** $q_\theta(a | s)$: MLP que produce μ y $\log \sigma$.
- **Estimador de recompensa** $g_\omega(s, a)$: MLP sobre el par estado-acción.
- **Potencial** $h_\phi(s)$: MLP sobre el estado.

Se implementó la forma estructurada:

$$f_{\omega, \phi}(s, a, s') = g_\omega(s, a) + \gamma h_\phi(s') - h_\phi(s)$$

24.6. Función discriminadora y objetivo

Siguiendo la Ecuación (11) del paper, el discriminador se define como:

$$D_\omega(s, a) = \frac{\exp(f_\omega(s, a))}{\exp(f_\omega(s, a)) + q_\theta(a | s)}$$

El entrenamiento usa:

- **Paso del discriminador**: maximiza $\mathbb{E}_{X_E}[\log D] + \mathbb{E}_{X_\pi}[\log(1 - D)]$.
- **Paso del generador (política)**: maximiza $\mathbb{E}_{X_\pi}[f_\omega(s, a) - \log q_\theta(a | s)]$ (Ecuación 12).

24.7. Proxy temporal

Los datos no incluyen transiciones temporales explícitas (s, s') . Para mantener la estructura MA-AIRL se aplicó un **proxy de un paso**:

$$s' = s$$

De esta forma se conserva la forma del potencial sin requerir secuencias completas. Esta decisión se documentó en el código como aproximación.

24.8. Métricas de entrenamiento

Durante el entrenamiento se reportan:

- **disc_loss**: pérdida del discriminador.
- **policy_loss**: pérdida de la política.
- **reward_mean**: promedio de f_ω en muestras expertas.

Se guardan en memoria para graficado y análisis posterior.

24.9. Exportación de parámetros a SUMO

Al finalizar, se obtienen parámetros por agente evaluando la política en el estado medio y des-normalizando a rangos SUMO. Se exporta un archivo:

- `mairl_vtypes.add.xml` con un `vType` por clúster.

Esto permite inyectar los parámetros aprendidos en simulaciones SUMO reales.

24.10. Integración en la UI

Se añadió un selector de algoritmo:

- **MA-AIRL (paper)**: entrenamiento adversarial con datos expertos.
- **CEM (legacy)**: método anterior basado en cross-entropy con SUMO.

También se agregó control de hiperparámetros (batch size, γ , *learning rates*, tamaño de red y dispositivo).

24.11. Archivos modificados

- `src/marl_core.py`: núcleo MA-AIRL (datos, redes, entrenamiento, exportación).
- `src/multi_agent_rl_app.py`: UI y flujo de entrenamiento MA-AIRL.

24.12. Limitaciones actuales y próximos pasos

- La implementación usa $s' = s$ por falta de secuencias temporales explícitas.
- Las acciones expertas son heurísticas derivadas de métricas agregadas.
- El siguiente paso recomendado es construir trayectorias reales desde SUMO o datos temporales para estimar s' y acciones reales.